



Wydział
Biologii

Wprowadzenie do głębokich architektur sieci neuronowych

Podstawowe sieci MLP

Wykład nr 2

dr Mateusz Draga

Zakład Hydrobiologii

matdra@amu.edu.pl

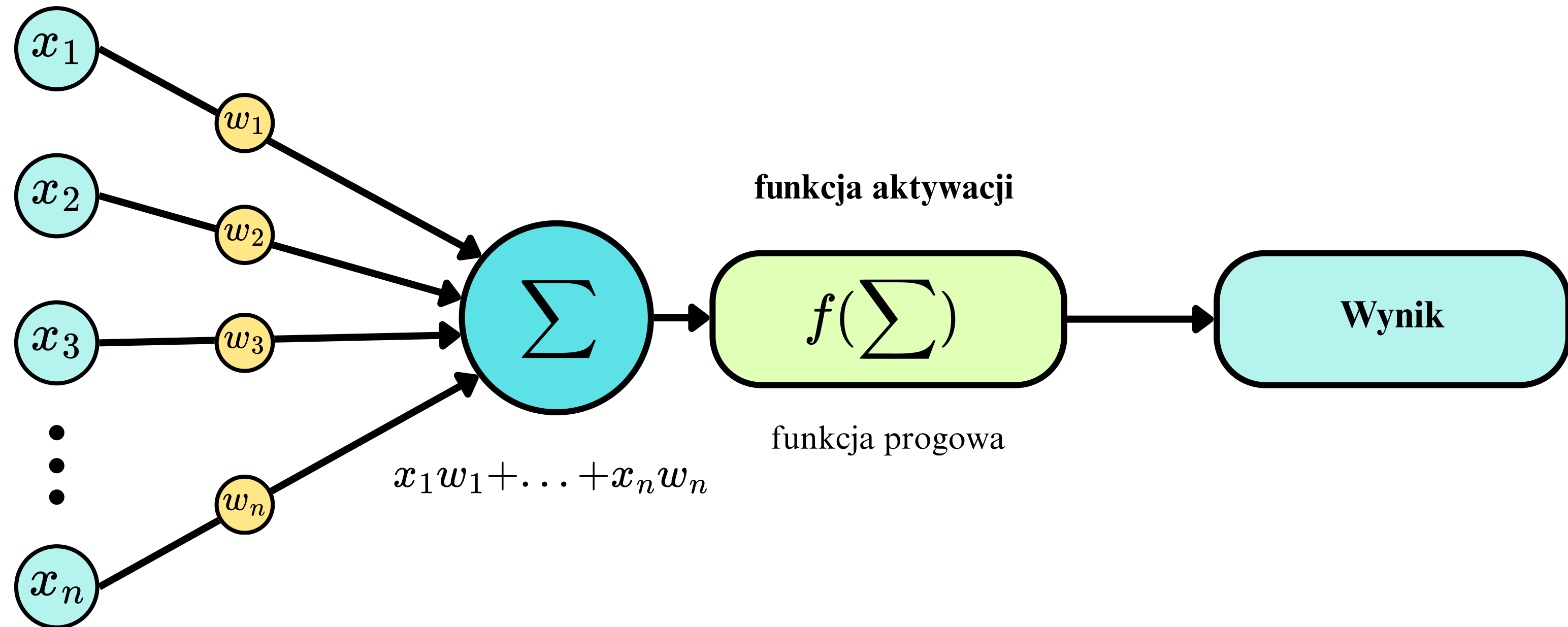
Perceptron - powtórka



Wydział
Biologii

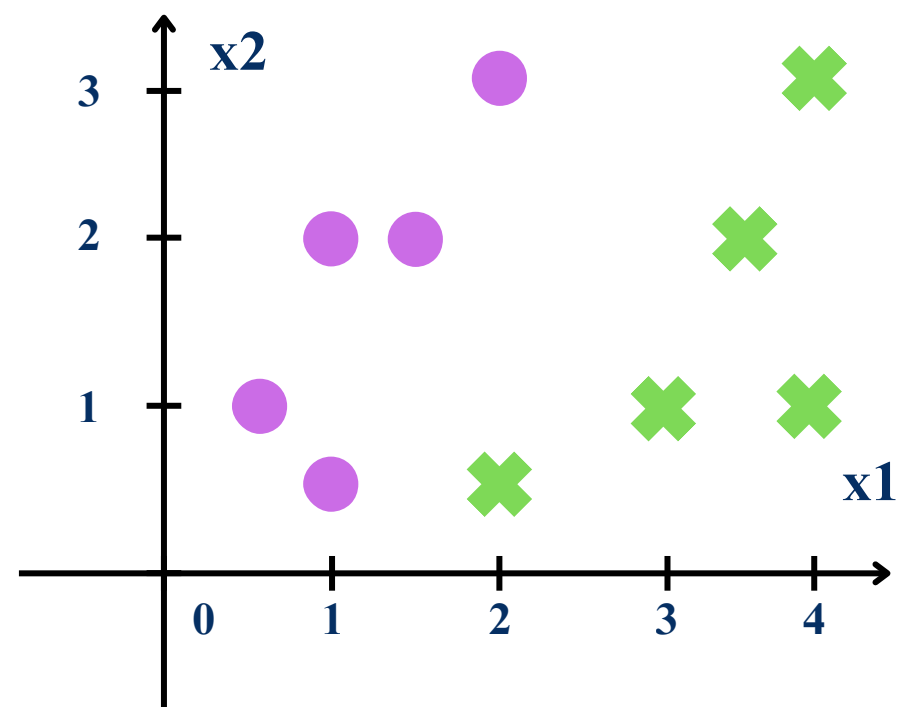
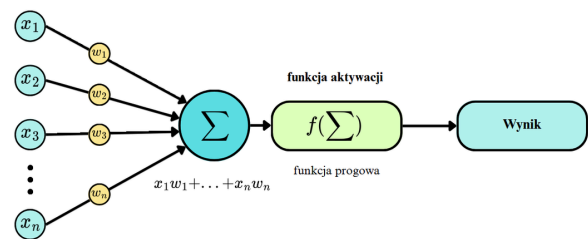
input

output

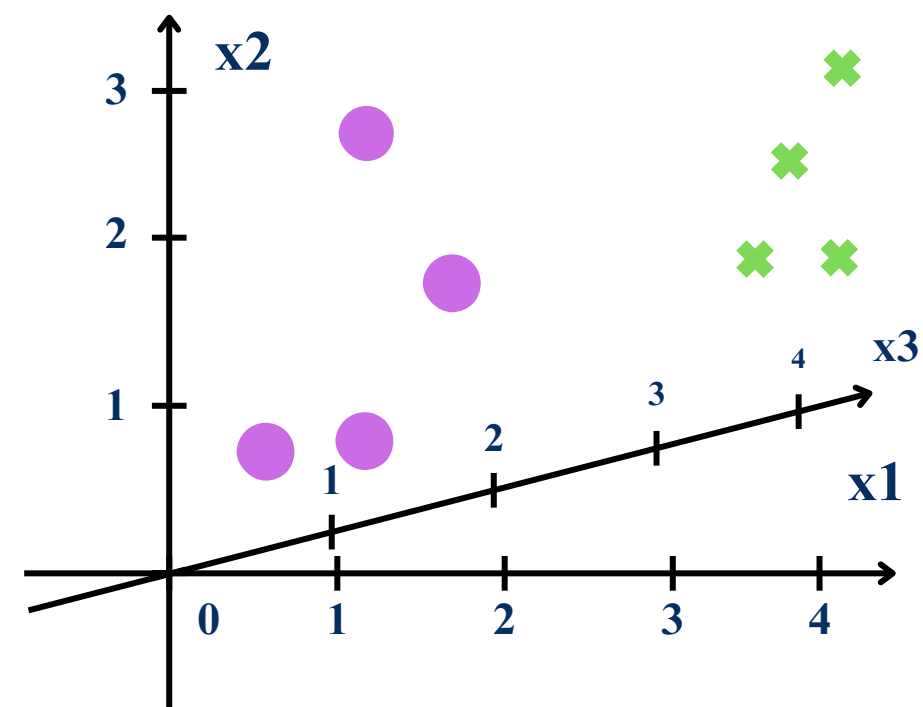


dane wejściowe

dane wyjściowe



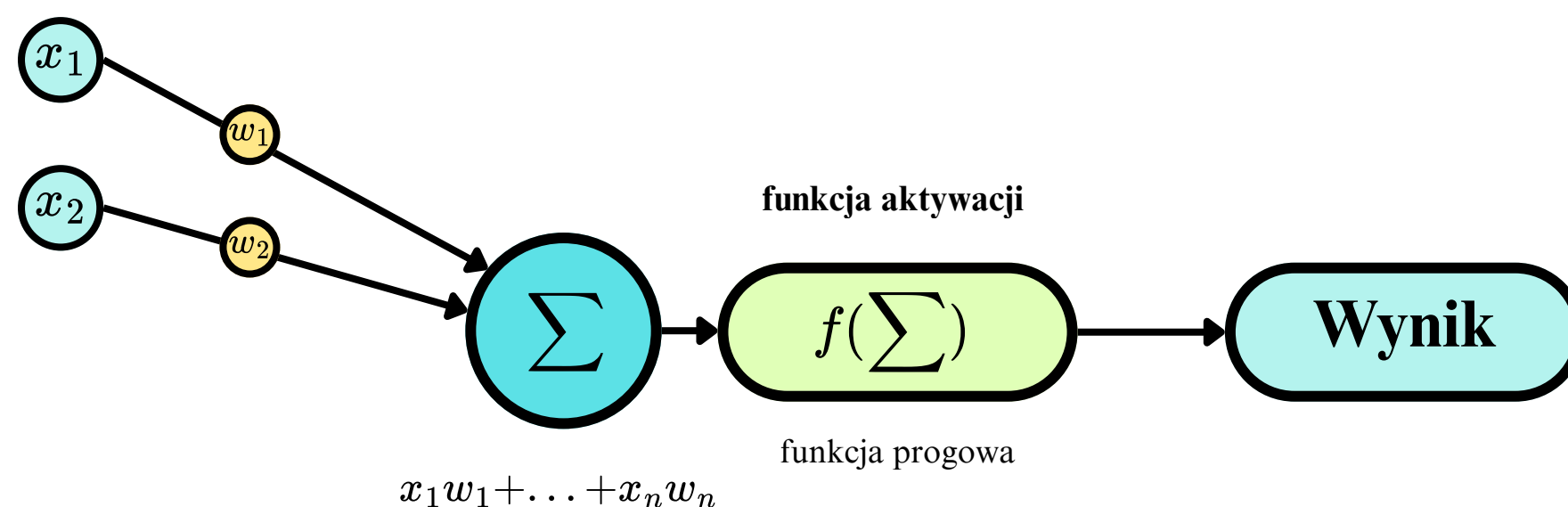
Input: z 2 nodów (x1 i x2)
Wynik: 2 klasy (0 i 1)



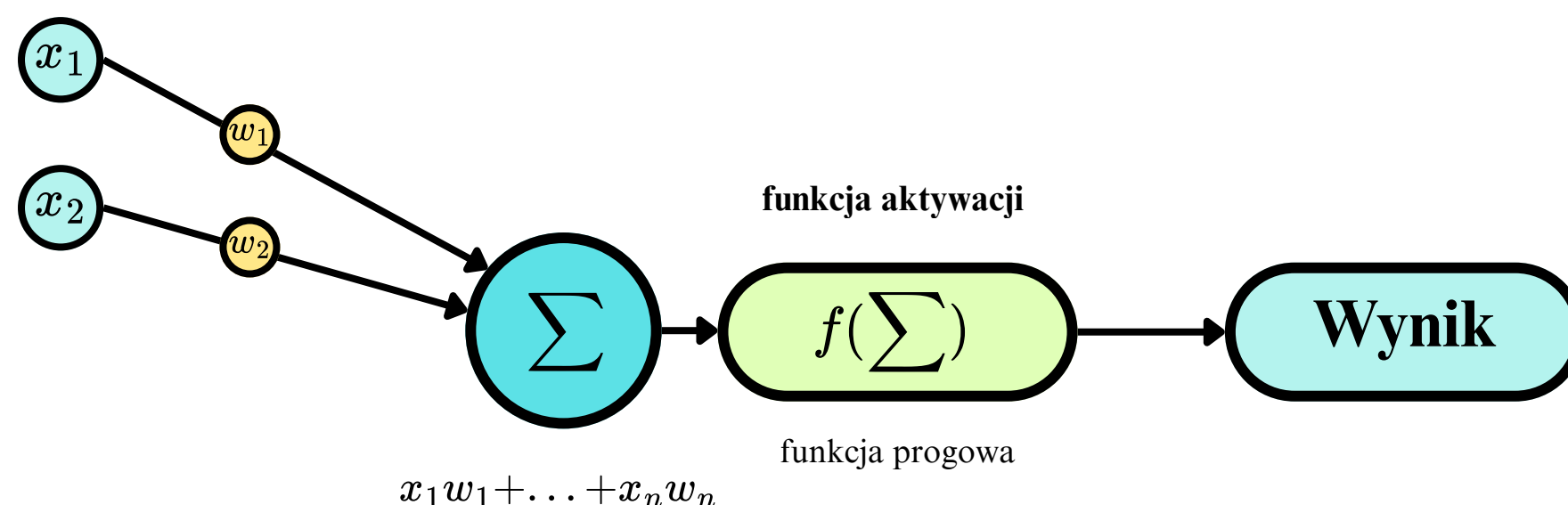
Input: z 3 nodów (x1, x2 i x3)
Wynik: 2 klasy (0 i 1)

Przekształcając odpowiednio otrzymane wartości **wag** i **biasów** naszych **wytrenowanych perceptronów**, możemy otrzymać **wzory na hiperprzestrzeń** rozdzielającą nasze dwie klasy.

$$y = f \left(bias + \sum_{i=1}^n x_i w_i \right)$$



Przekształcając odpowiednio otrzymane wartości **wag** i **biasów** naszych **wytrenowanych perceptronów**, możemy otrzymać **wzory na hiperprzestrzeń** rozdzielającą nasze dwie klasy.



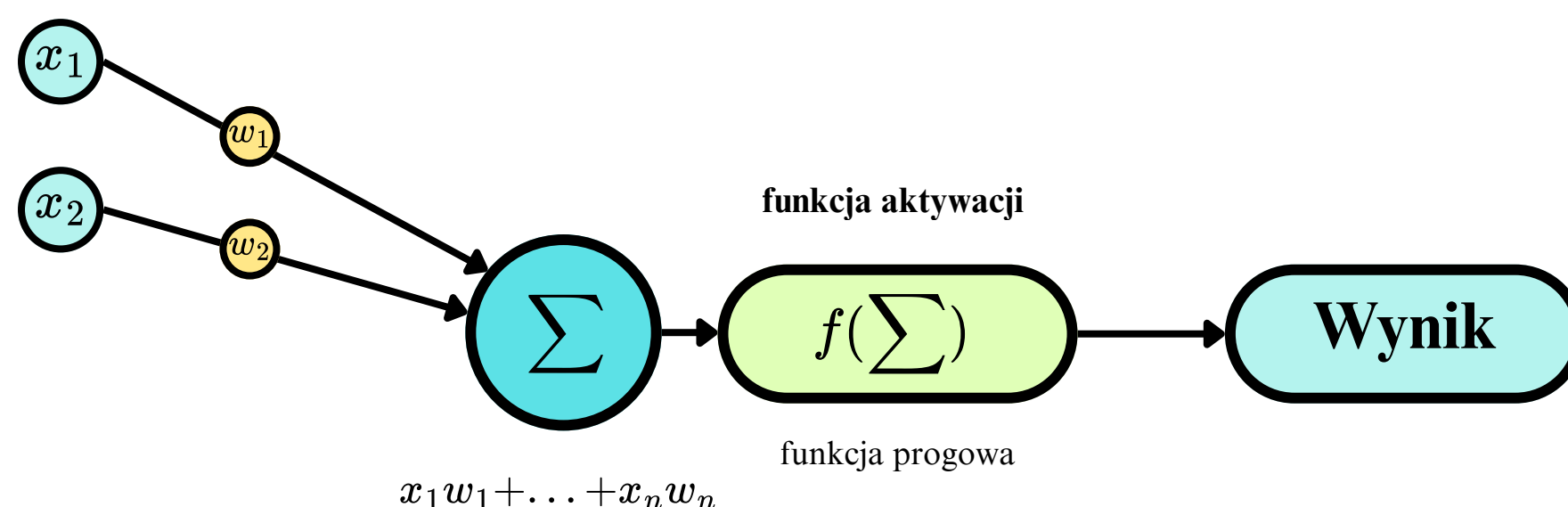
$$y = f \left(bias + \sum_{i=1}^n x_i w_i \right)$$

Dla 2 danych wejściowych (x_1 i x_2):

$$0 = w_1 x_1 + w_2 x_2 + b$$

$$x_2 = -\frac{w_1}{w_2} x_1 - \frac{1}{w_2} b$$

Przekształcając odpowiednio otrzymane wartości **wag** i **biasów** naszych **wytrenowanych perceptronów**, możemy otrzymać **wzory na hiperprzestrzeń** rozdzielającą nasze dwie klasy.



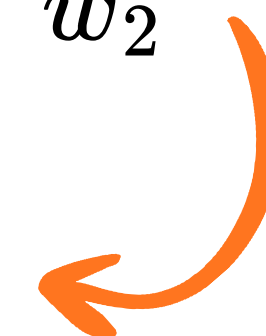
$$y = f \left(bias + \sum_{i=1}^n x_i w_i \right)$$

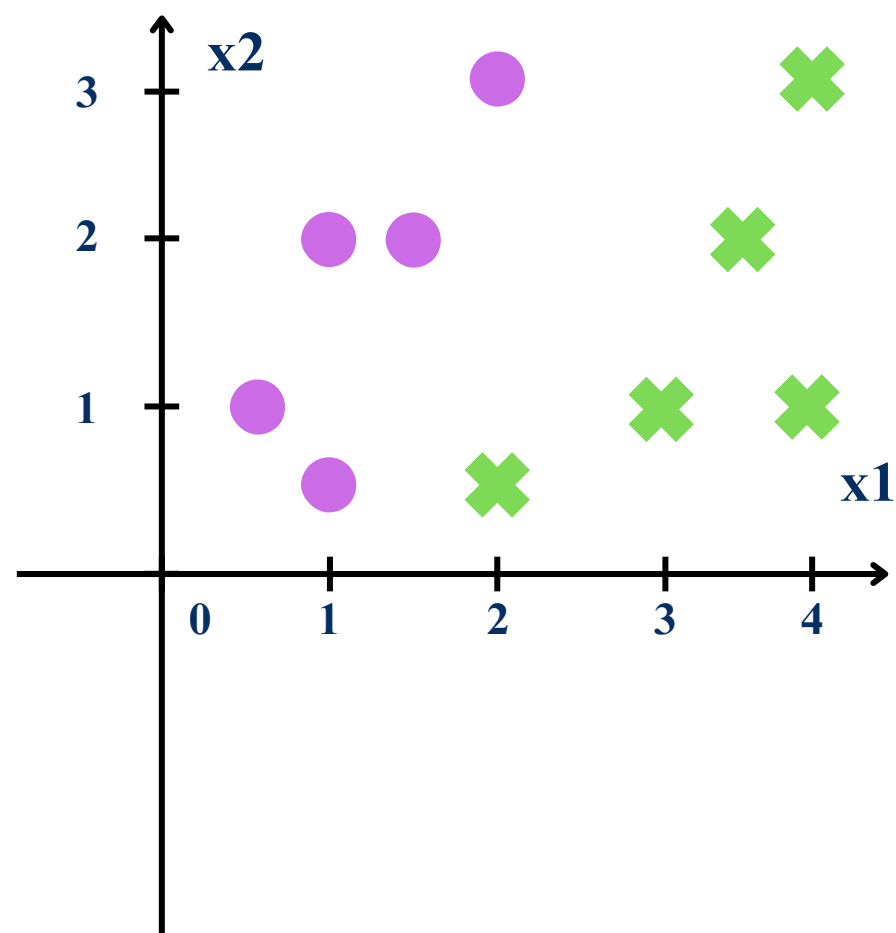
Dla 2 danych wejściowych (x1 i x2):

$$0 = w_1 x_1 + w_2 x_2 + b$$

$$x_2 = -\frac{w_1}{w_2} x_1 - \frac{1}{w_2} b$$

$$y = ax + b$$





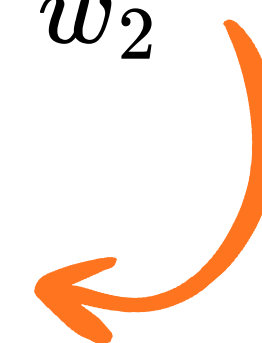
Input: z 2 nodów (x1 i x2)

Wynik: 2 klasy (0 i 1)

$$0 = w_1 x_1 + w_2 x_2 + b$$

$$x_2 = -\frac{w_1}{w_2} x_1 - \frac{1}{w_2} b$$

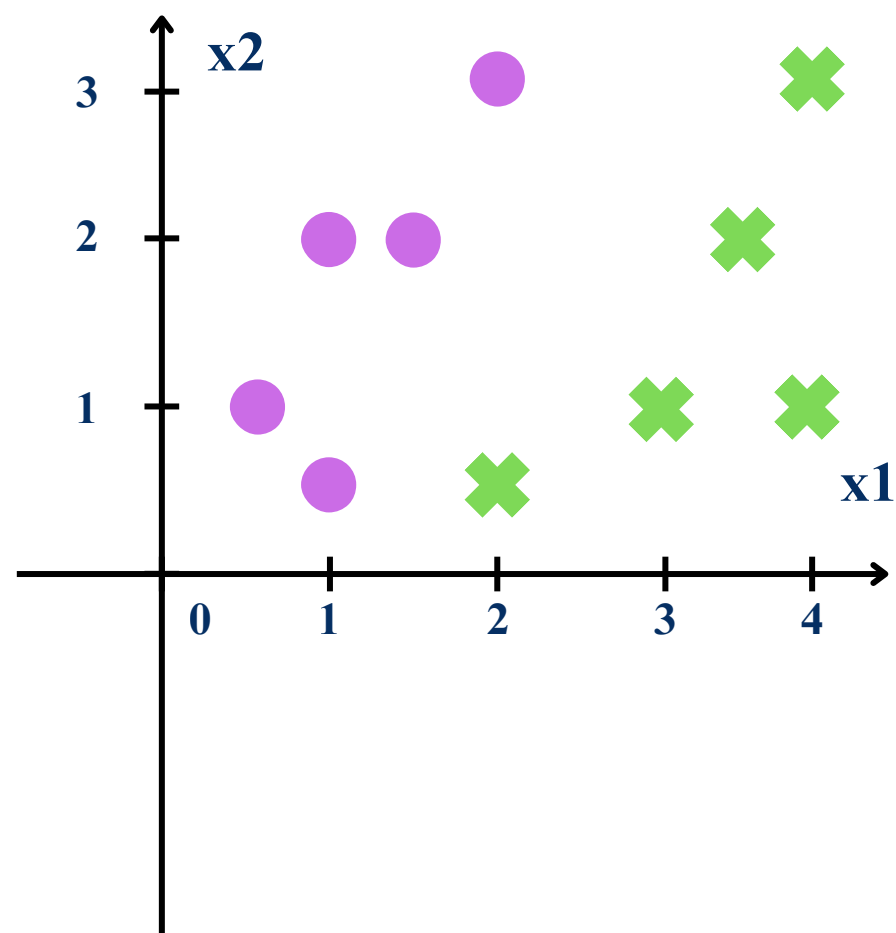
$$y = ax + b$$



Wynik treningu:

$$w_1 = 0.1 \quad w_2 = -0.05$$

$$b = -0.1$$



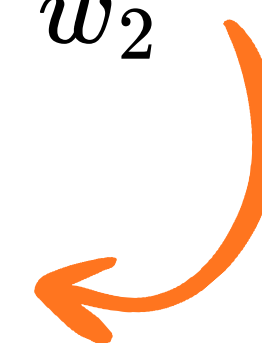
Input: z 2 nodów (x1 i x2)

Wynik: 2 klasy (0 i 1)

$$0 = w_1 x_1 + w_2 x_2 + b$$

$$x_2 = -\frac{w_1}{w_2} x_1 - \frac{1}{w_2} b$$

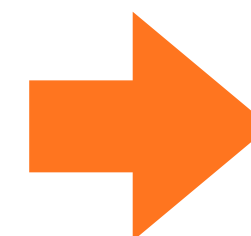
$$y = ax + b$$



Wynik treningu:

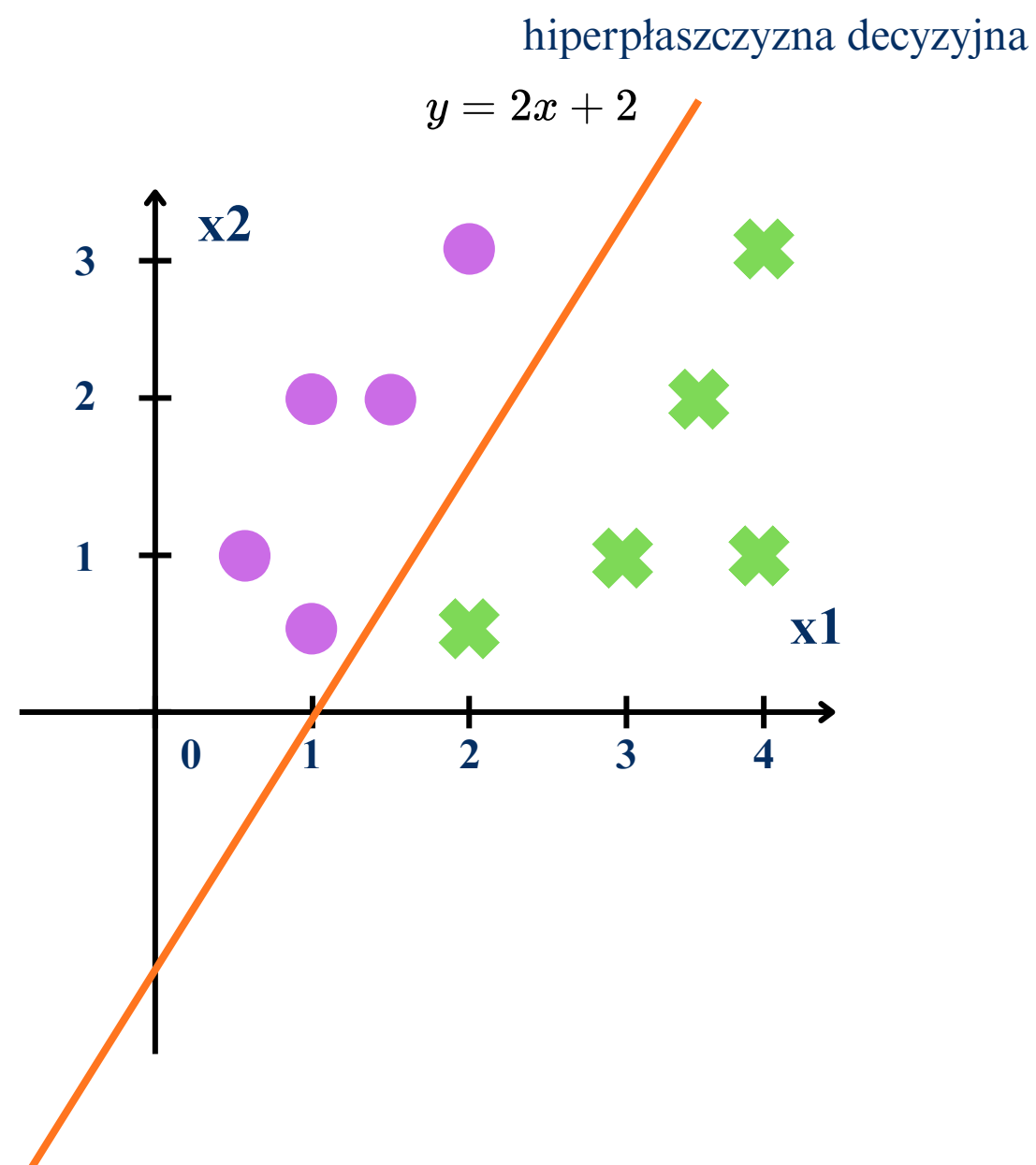
$$w_1 = 0.1 \quad w_2 = -0.05$$

$$b = -0.1$$



Po podstawieniu daje nam:

$$y = 2x + 2$$

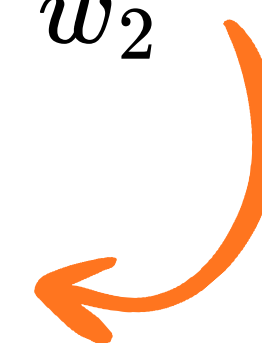


Input: z 2 nodów (x_1 i x_2)
Wynik: 2 klasy (0 i 1)

$$0 = w_1 x_1 + w_2 x_2 + b$$

$$x_2 = -\frac{w_1}{w_2} x_1 - \frac{1}{w_2} b$$

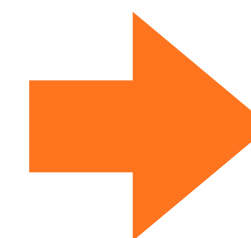
$$y = ax + b$$



Wynik treningu:

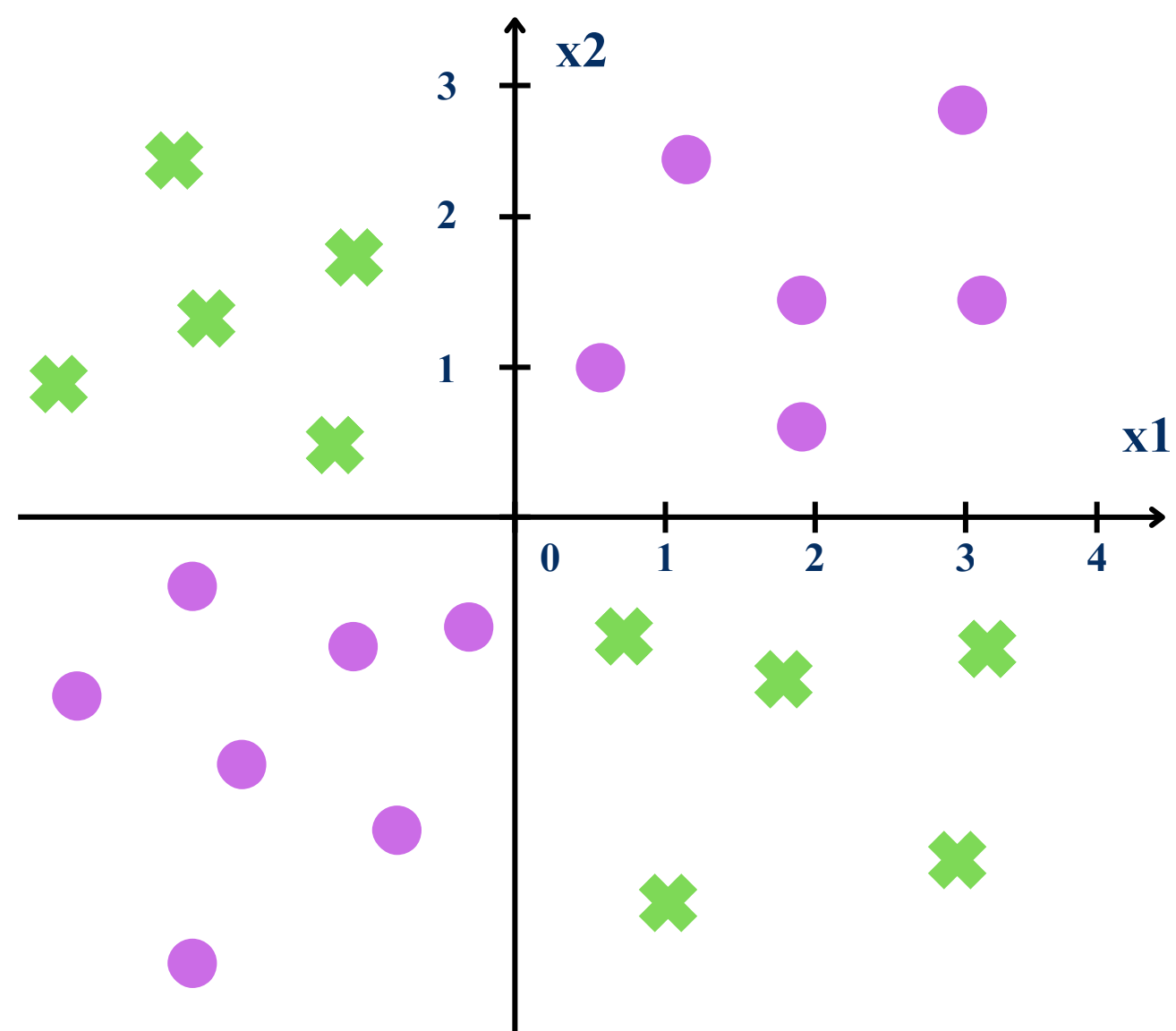
$$w_1 = 0.1 \quad w_2 = -0.05$$

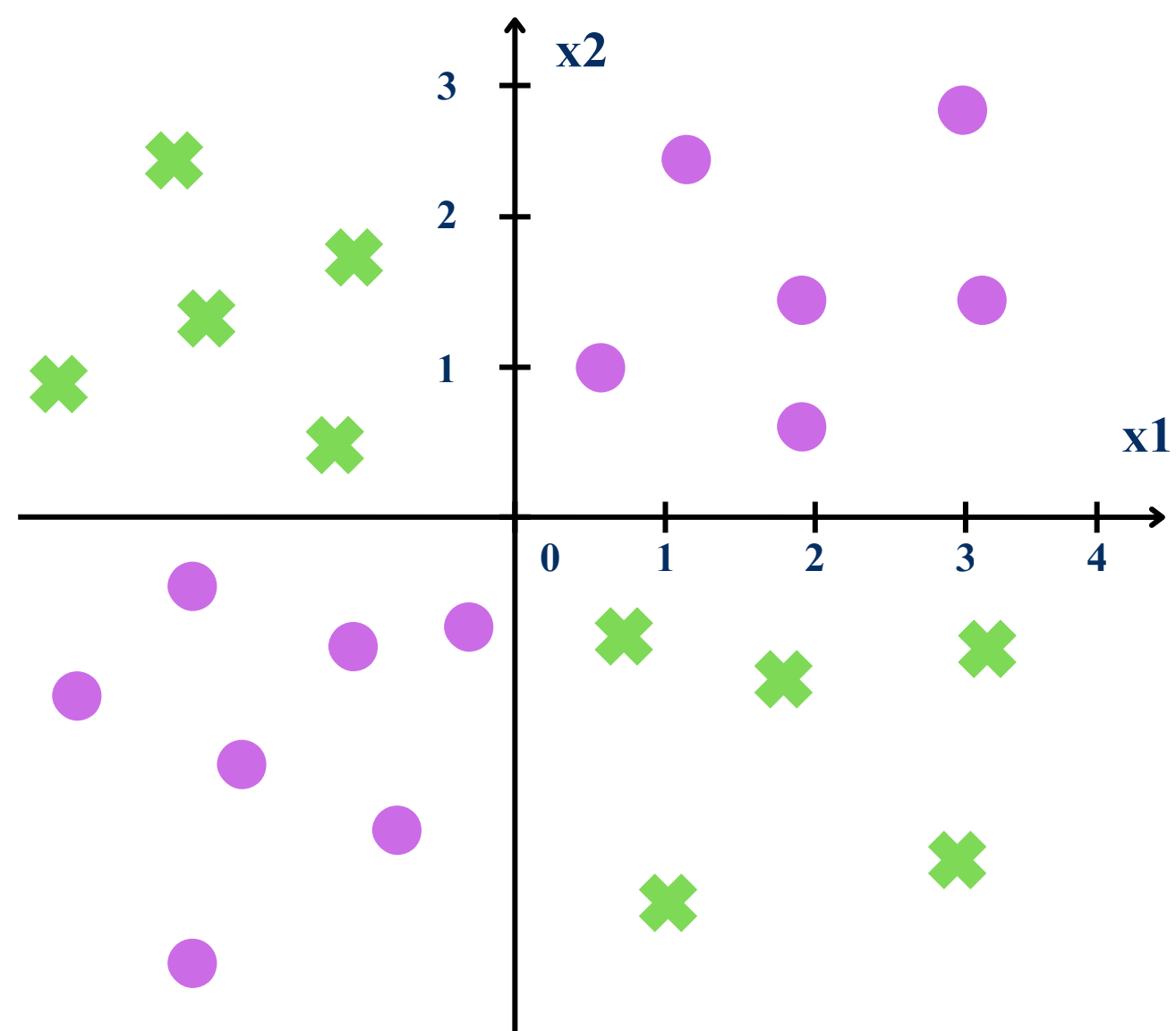
$$b = -0.1$$



Po podstawieniu daje nam:

$$y = 2x + 2$$

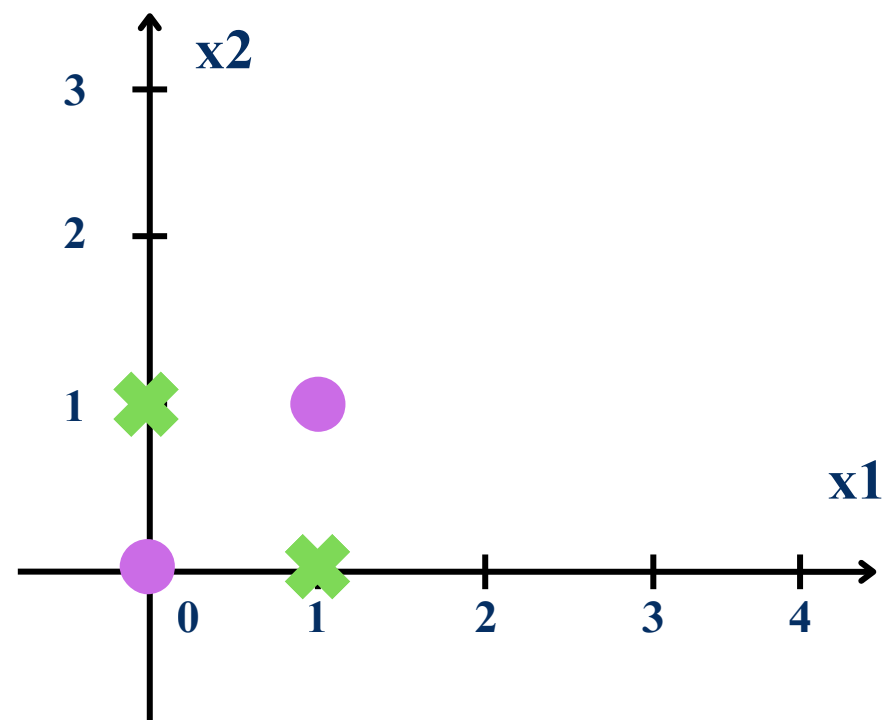




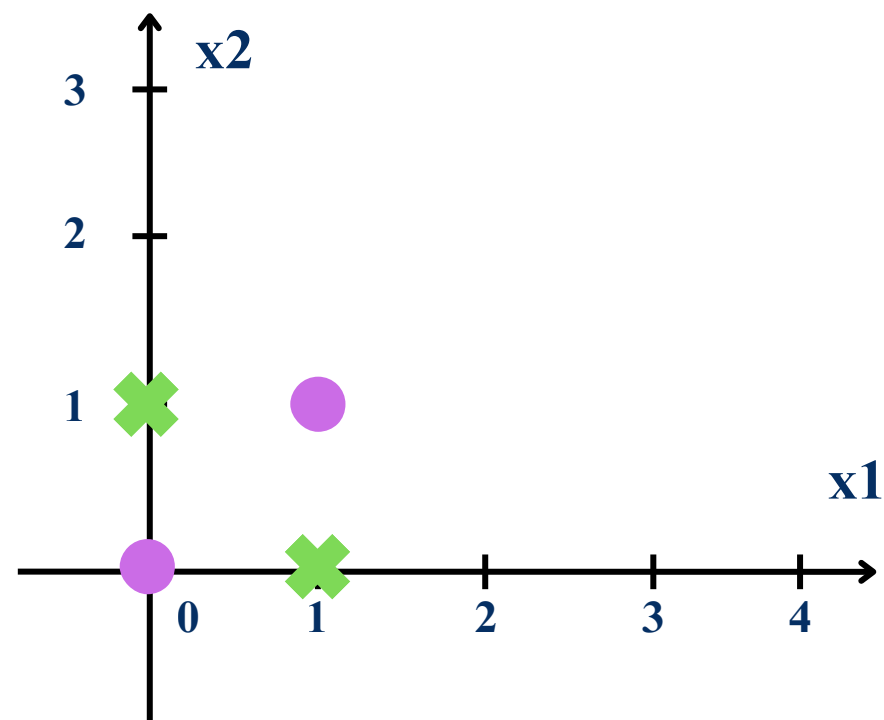
Problemy typu XOR



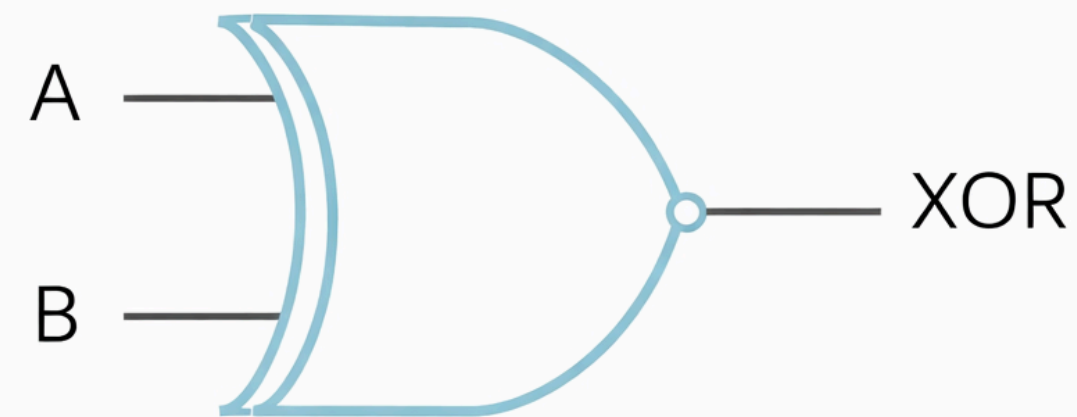
Wydział
Biologii



Problemy typu XOR

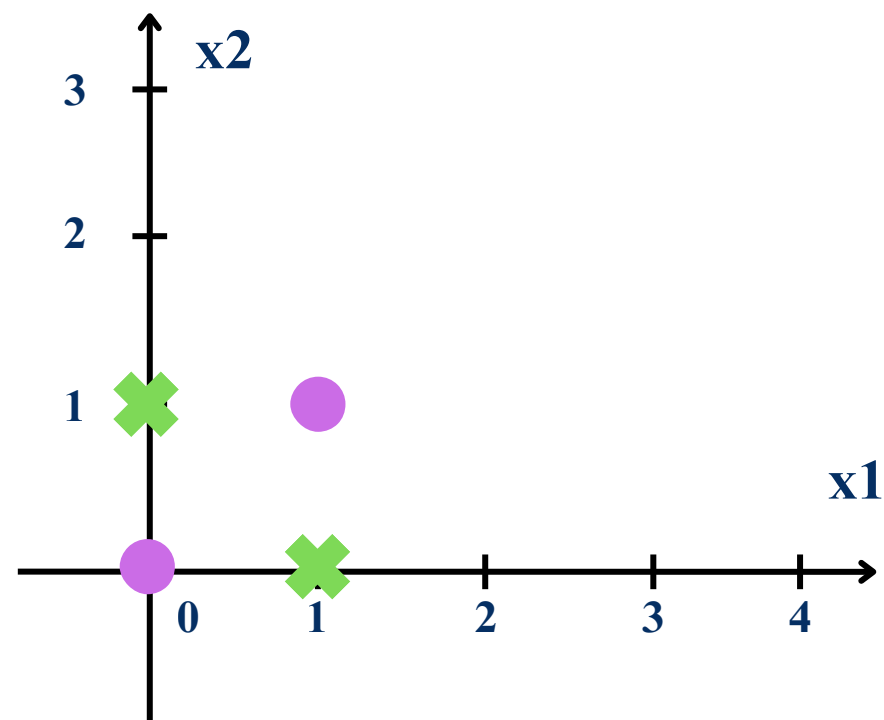


$$X = A \oplus B$$

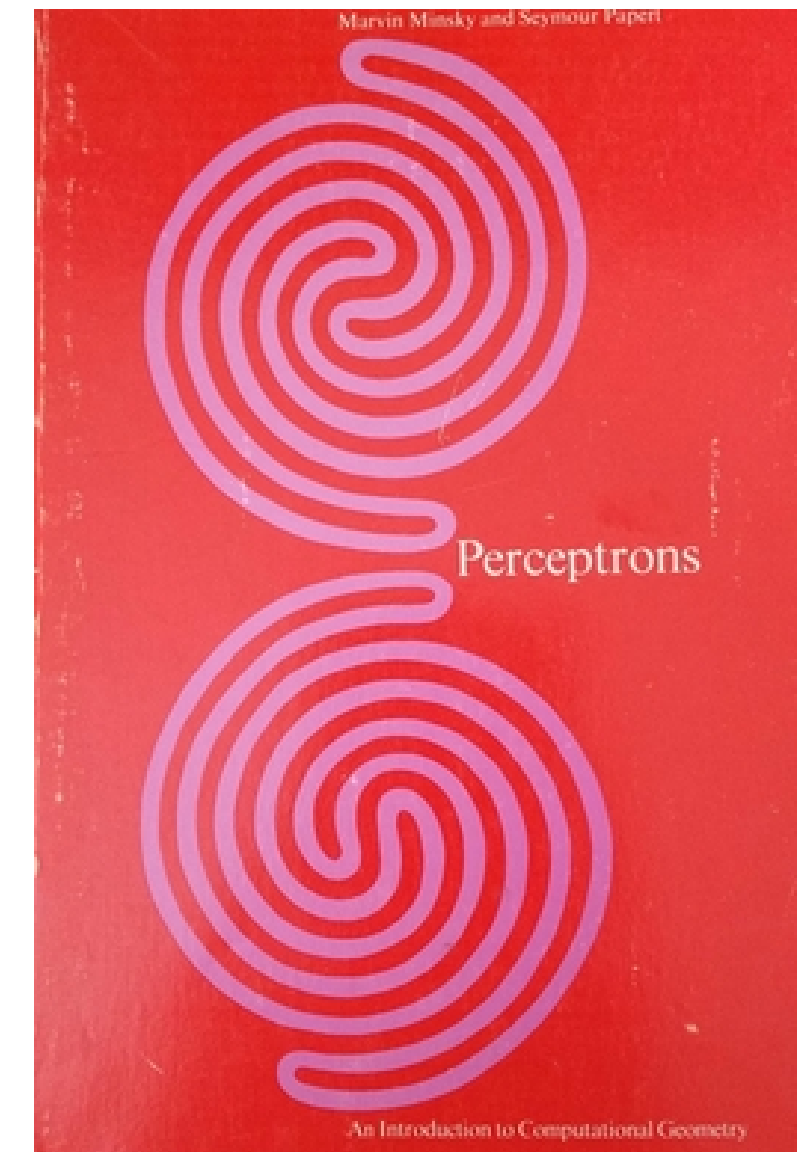


A	B	XOR
0	0	0
0	1	1
1	0	1
1	1	0

Problemy typu XOR



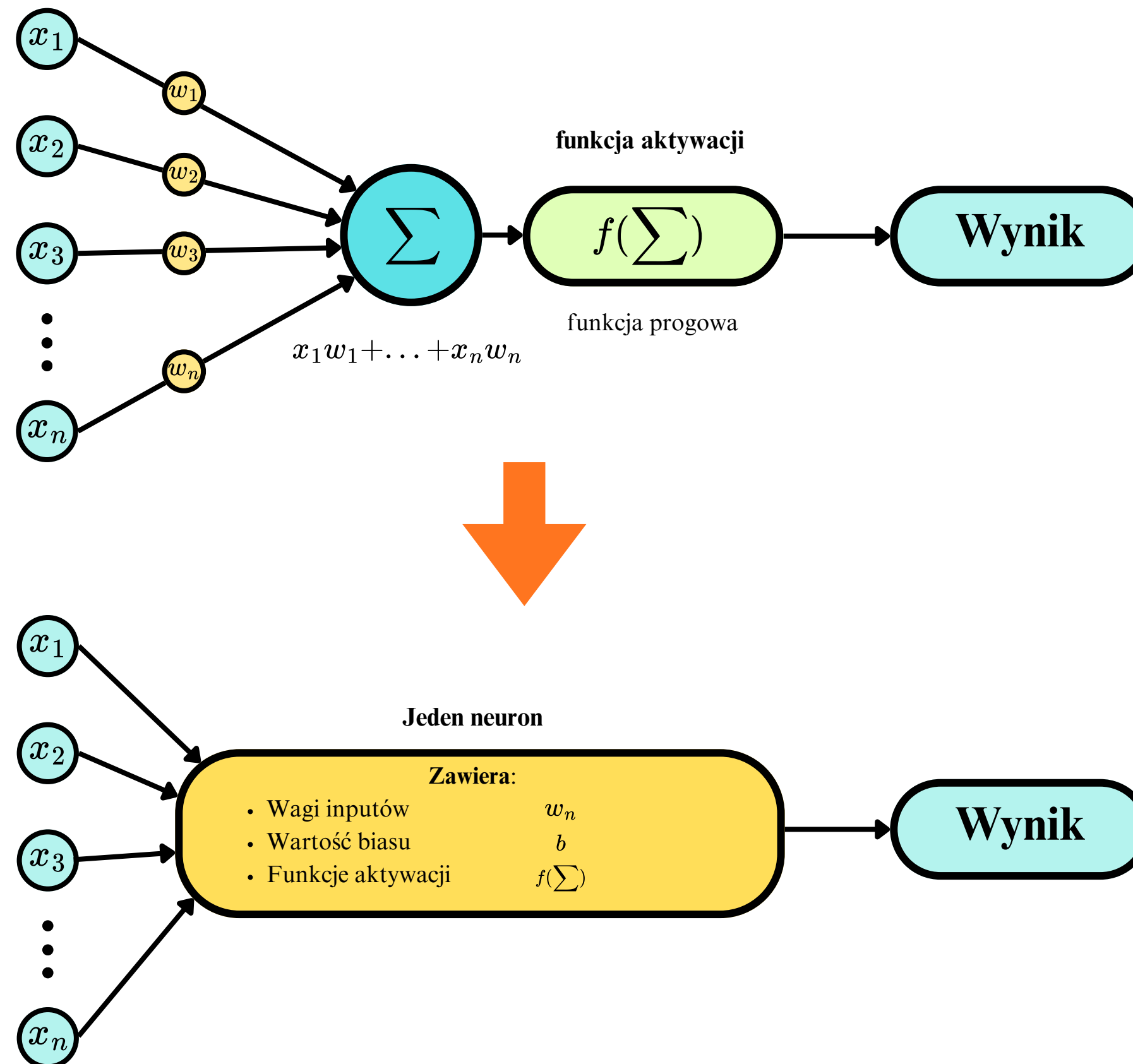
Marvin Minsky i Seymour Papert

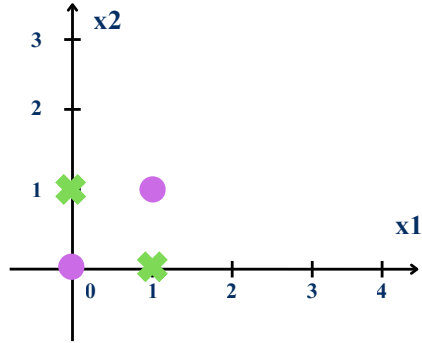


Sieci typu MLP



Wydział
Biologii





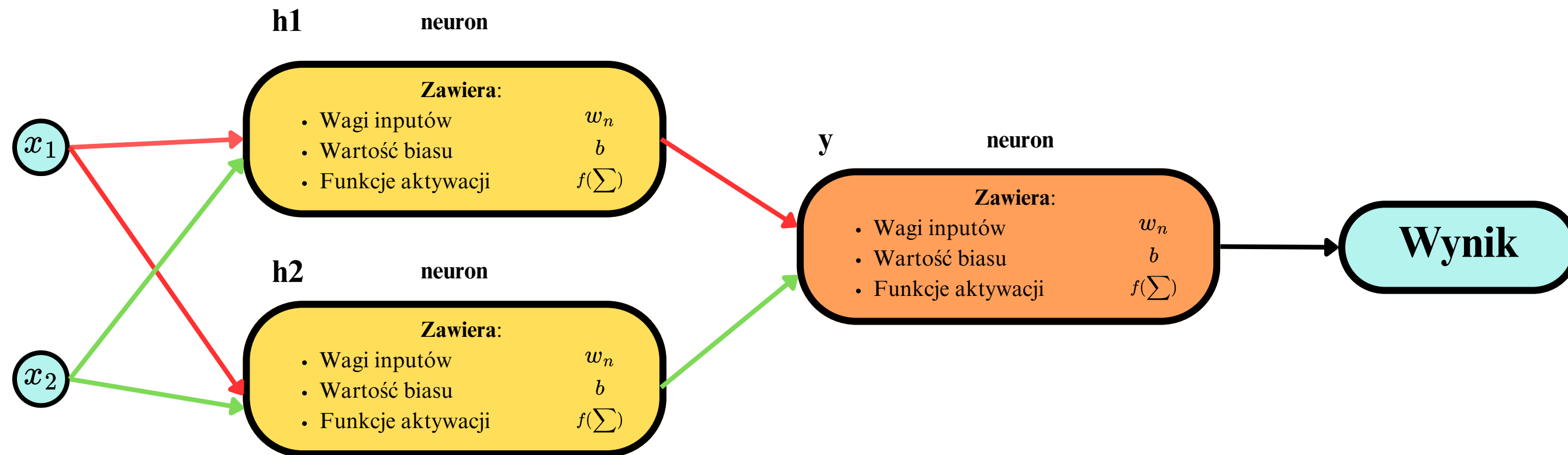
Sieci typu MLP

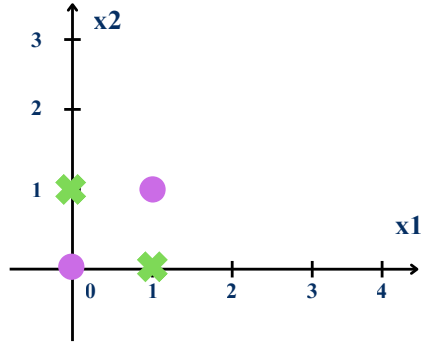
input

Warstwa ukryta

Neuron wyjściowy

output





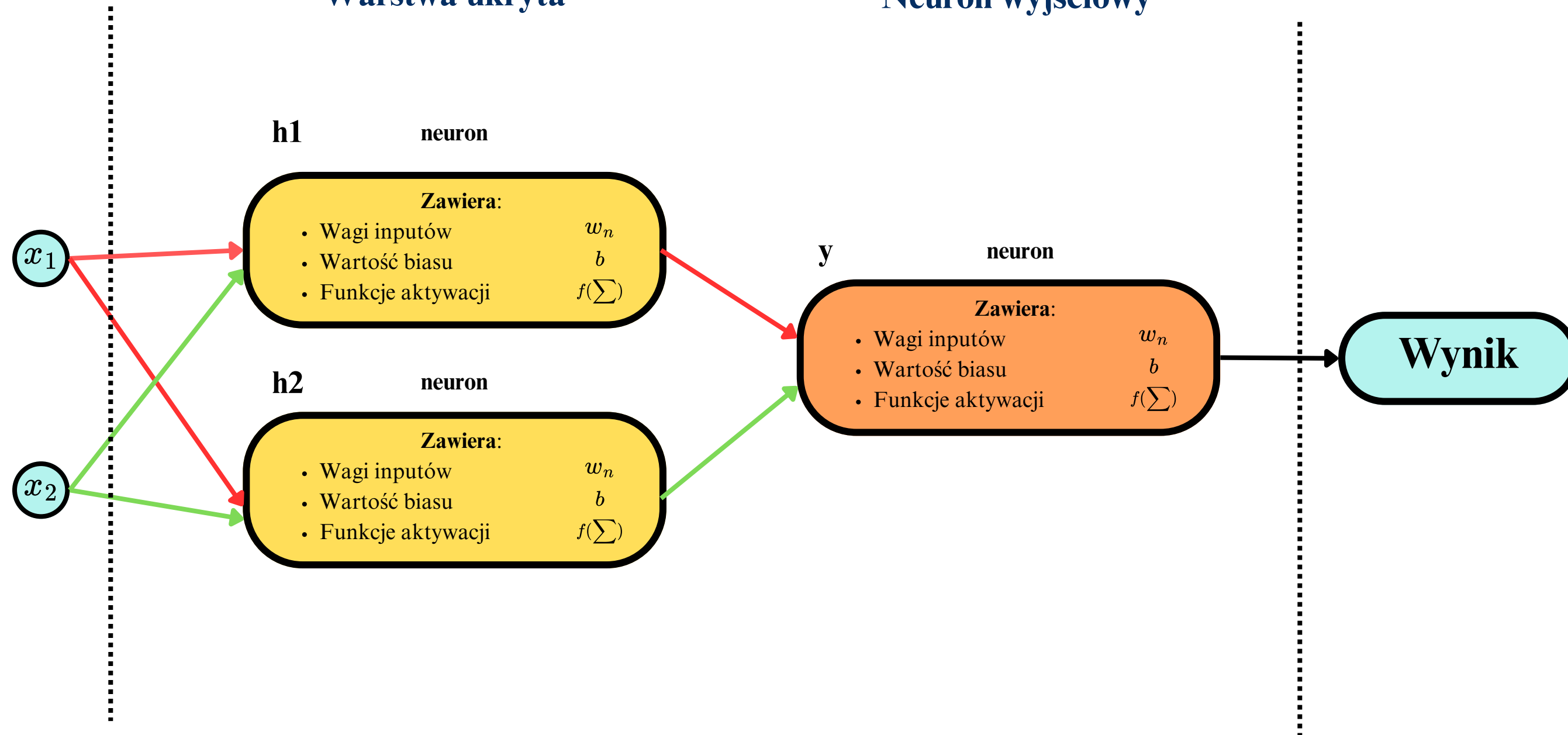
Sieci typu MLP

input

Warstwa ukryta

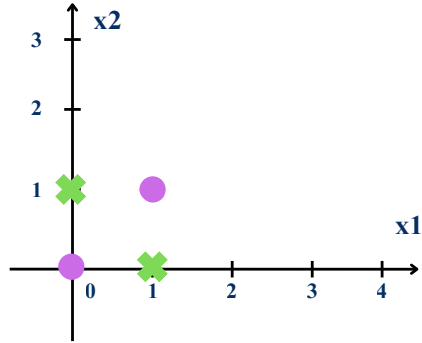
Neuron wyjściowy

output



Budowa bardzo prostej sieci neuronowej typu MLP:

- **Input:** x_1 i x_2
- **Warstwa ukryta** (dwa neurony)
- **Neuron wyjścia**
- **Output:** 0 lub 1



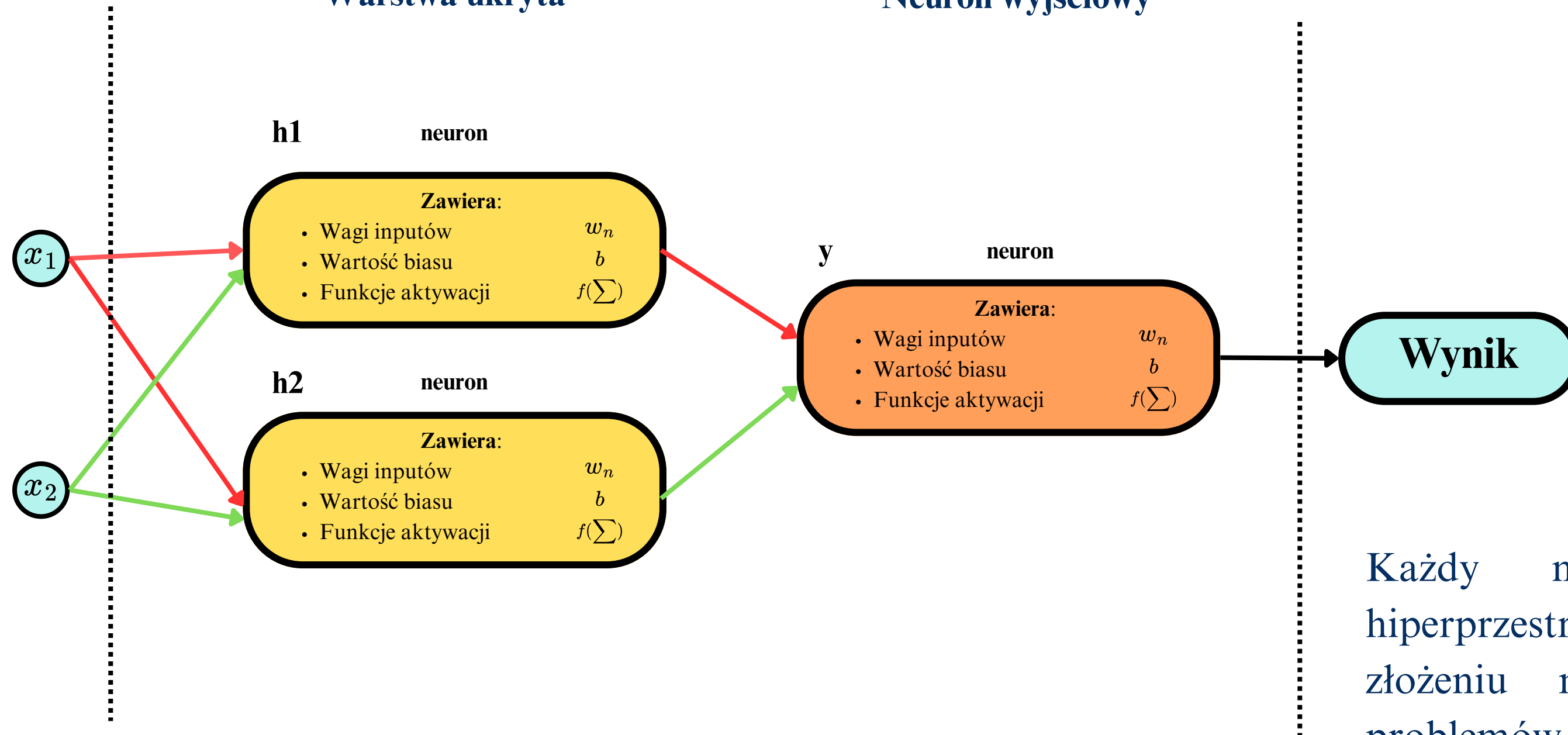
Sieci typu MLP

input

Warstwa ukryta

Neuron wyjściowy

output



Budowa bardzo prostej sieci neuronowej typu MLP:

- **Input:** x_1 i x_2
- **Warstwa ukryta** (dwa neurony)
- **Neuron wyjścia**
- **Output:** 0 lub 1

Każdy nowy neuron to nowa hiperprzestrzeń decyzyjna! Dzięki ich złożeniu możliwe jest rozwiązywanie problemów liniowo nieseparowalnych.

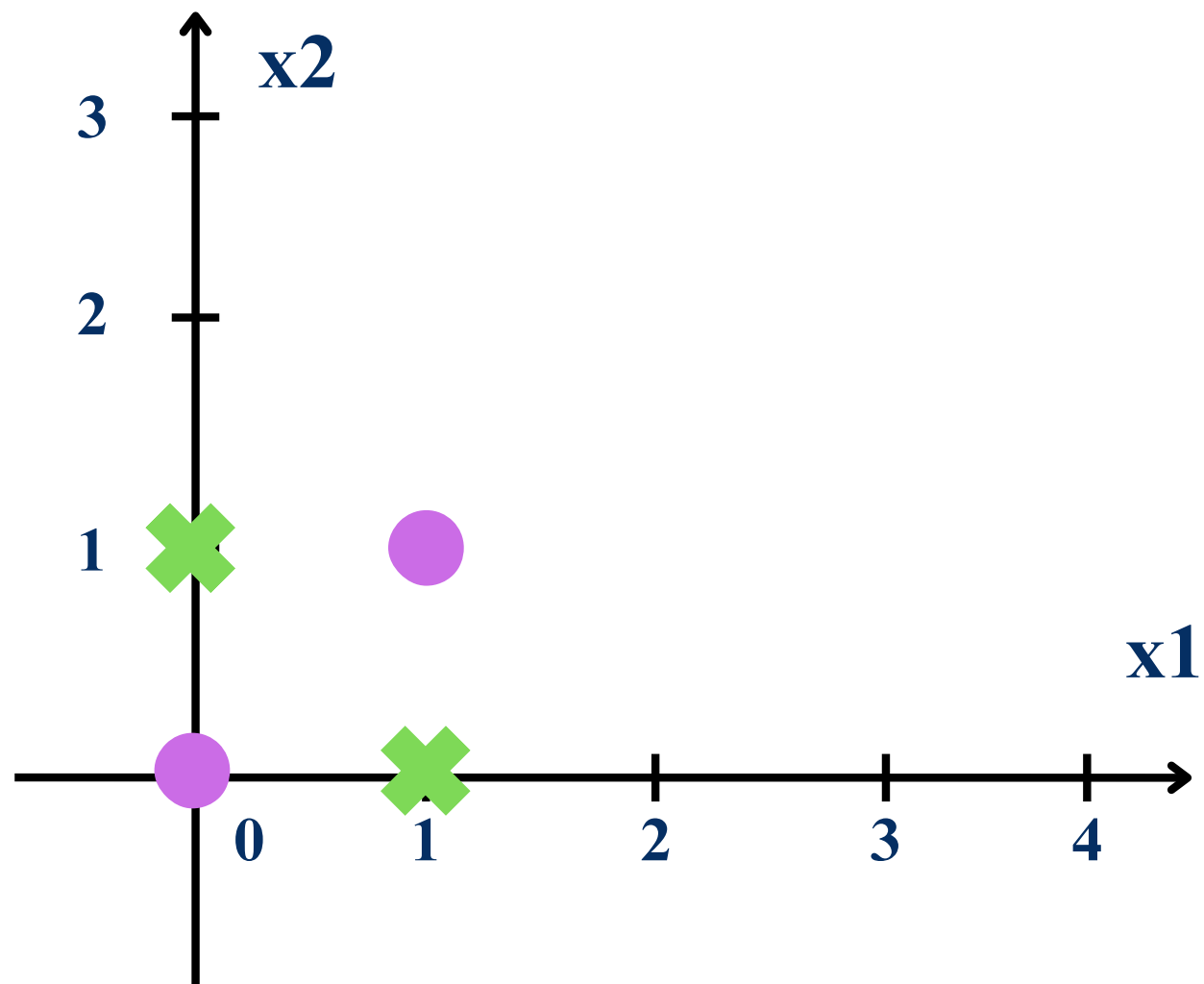
Sieci typu MLP



Wydział
Biologii

Schemat pracy sieci

$$x_1, x_2 \rightarrow h_1, h_2 \rightarrow y \rightarrow \text{wynik}$$



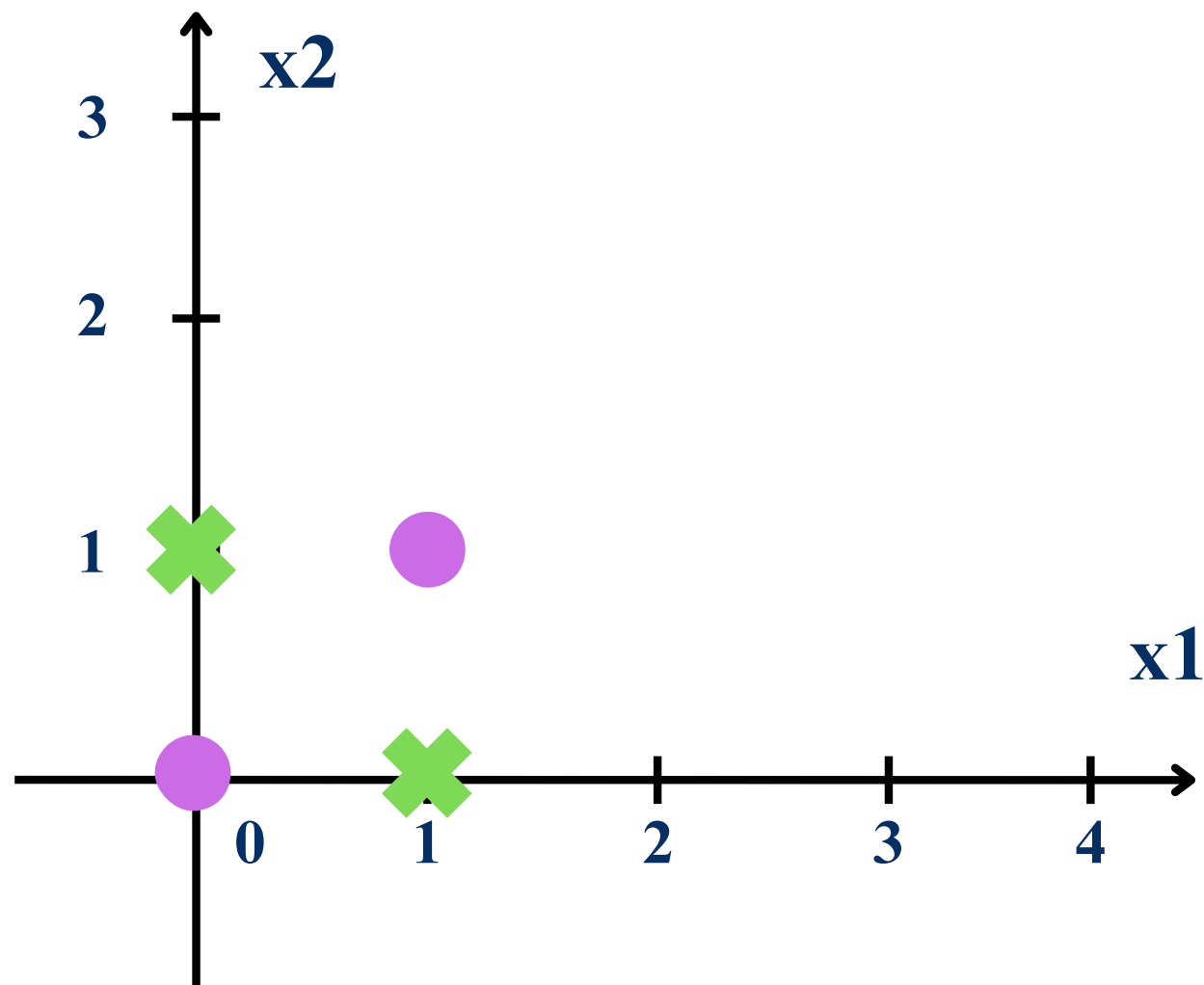
Sieci typu MLP



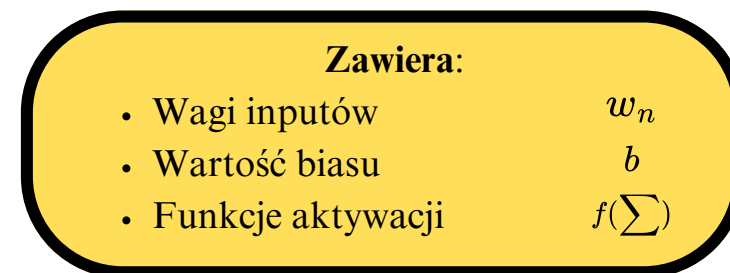
Wydział
Biologii

Schemat pracy sieci

$$x_1, x_2 \rightarrow h_1, h_2 \rightarrow y \rightarrow \text{wynik}$$

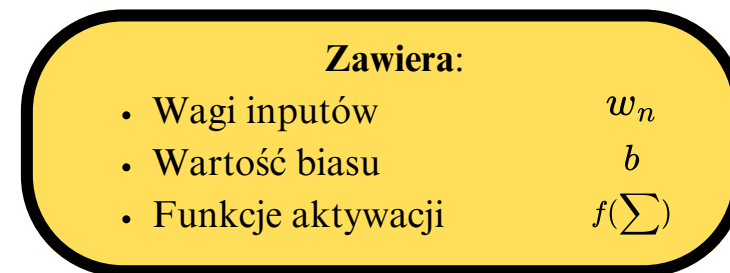


h1 neuron



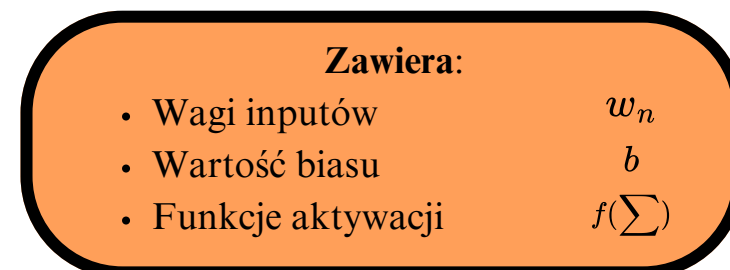
$$\begin{aligned} w_1 &= 1 \\ w_2 &= 1 \\ b &= -0.5 \end{aligned}$$

h2 neuron



$$\begin{aligned} w_1 &= 1 \\ w_2 &= 1 \\ b &= -1.5 \end{aligned}$$

y neuron



$$\begin{aligned} w_1 &= 1 \\ w_2 &= -2 \\ b &= -0.5 \end{aligned}$$

Wartości po treningu

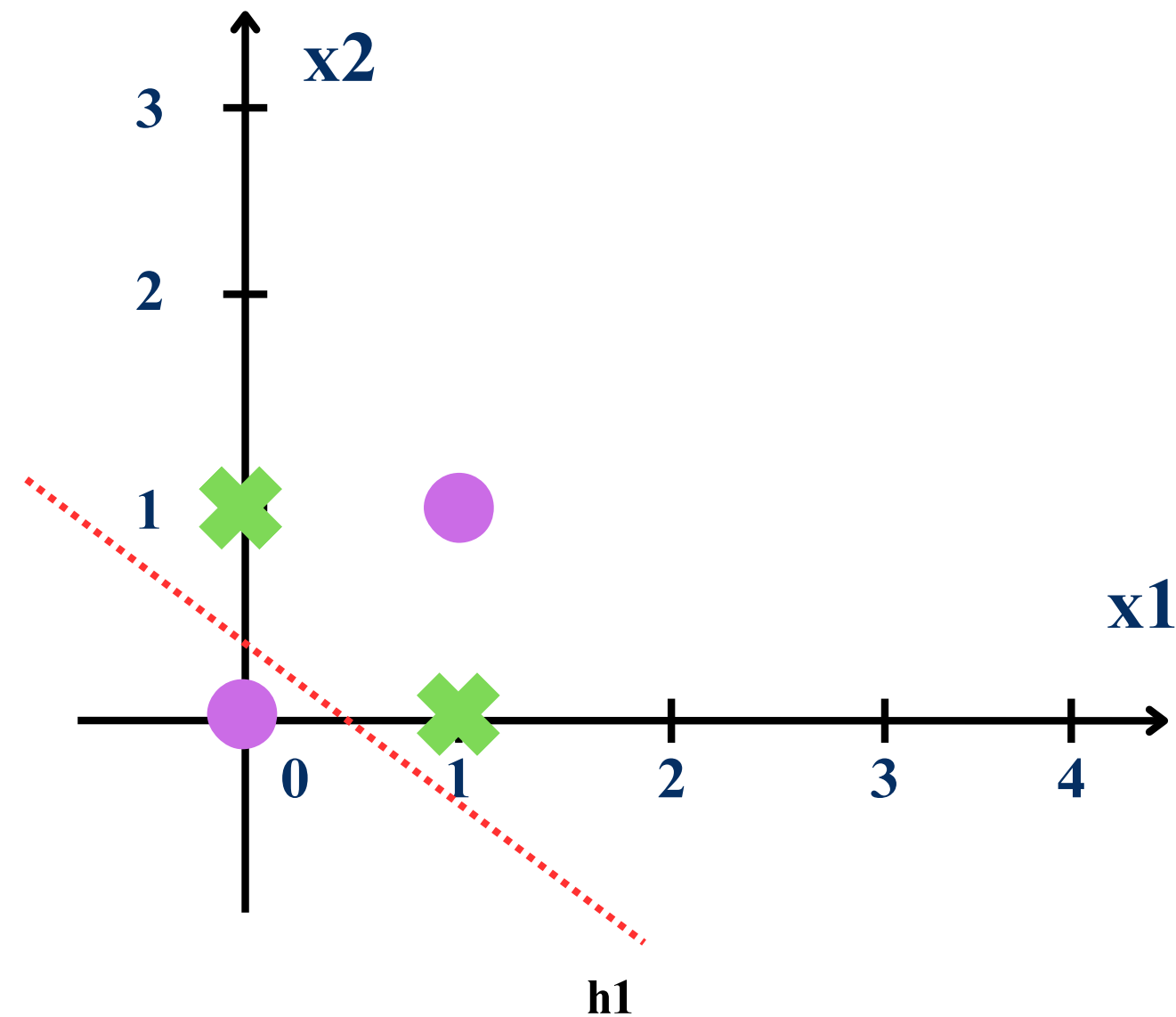
Sieci typu MLP



Wydział
Biologii

Schemat pracy sieci

$$x_1, x_2 \rightarrow h_1, h_2 \rightarrow y \rightarrow \text{wynik}$$



h1 neuron

Zawiera:

- Wagi inputów w_n
- Wartość biasu b
- Funkcje aktywacji $f(\sum)$

$$w_1 = 1$$
$$w_2 = 1$$
$$b = -0.5$$

Czy cokolwiek się aktywowało? (**OR**)

$$y = -x + 0.5$$

h2 neuron

Zawiera:

- Wagi inputów w_n
- Wartość biasu b
- Funkcje aktywacji $f(\sum)$

$$w_1 = 1$$
$$w_2 = 1$$
$$b = -1.5$$



y neuron

Zawiera:

- Wagi inputów w_n
- Wartość biasu b
- Funkcje aktywacji $f(\sum)$

$$w_1 = 1$$
$$w_2 = -2$$
$$b = -0.5$$

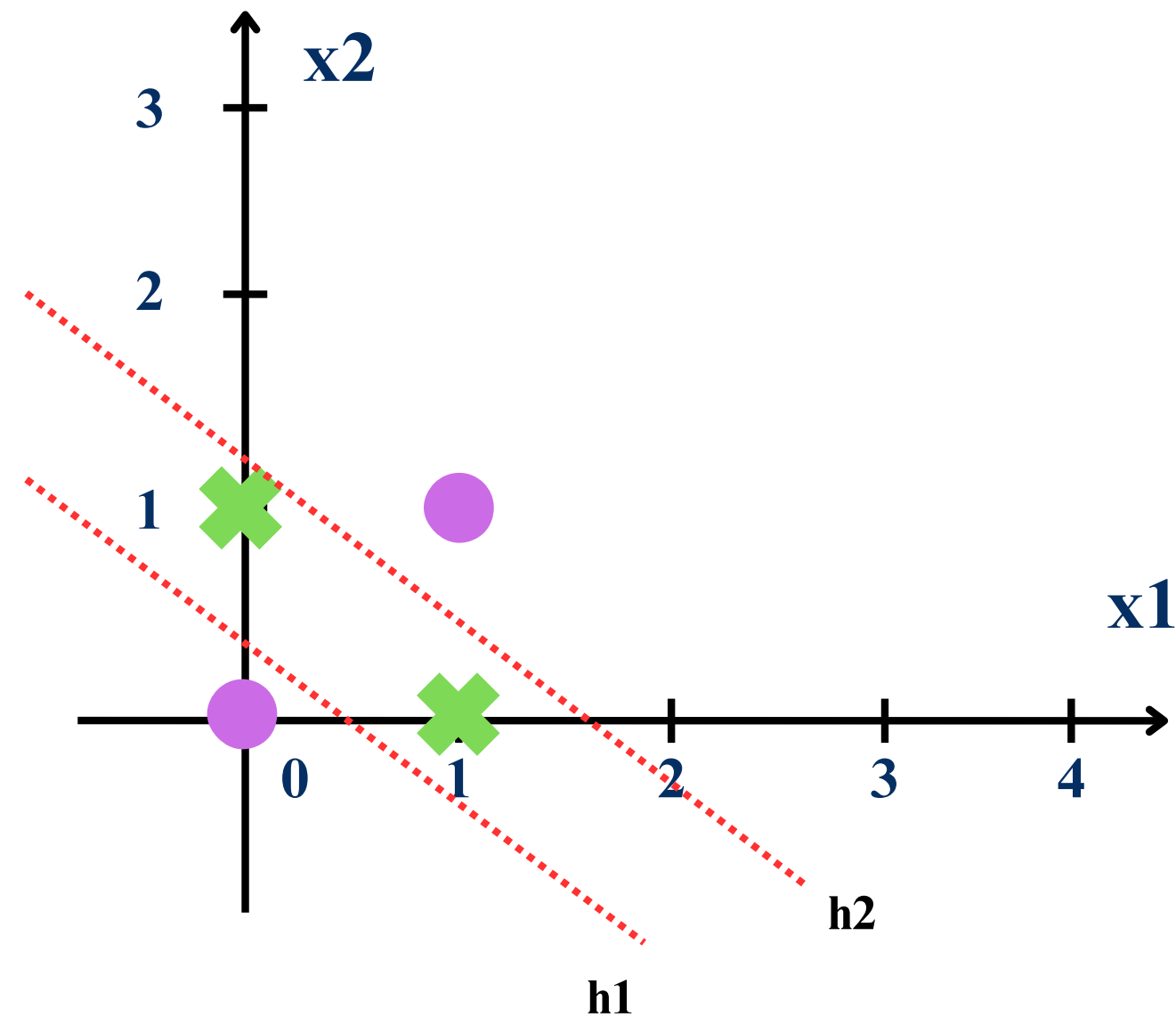
Sieci typu MLP



Wydział
Biologii

Schemat pracy sieci

$$x_1, x_2 \rightarrow h_1, h_2 \rightarrow y \rightarrow \text{wynik}$$



h1 neuron

Zawiera:

- Wagi inputów w_n
- Wartość biasu b
- Funkcje aktywacji $f(\sum)$

$w_1 = 1$
 $w_2 = 1$
 $b = -0.5$

Czy cokolwiek się aktywowało? (**OR**)

$$y = -x + 0.5$$

h2 neuron

Zawiera:

- Wagi inputów w_n
- Wartość biasu b
- Funkcje aktywacji $f(\sum)$

$w_1 = 1$
 $w_2 = 1$
 $b = -1.5$

Czy wszystko się aktywowało? (**AND**)

$$y = -x + 1.5$$



y neuron

Zawiera:

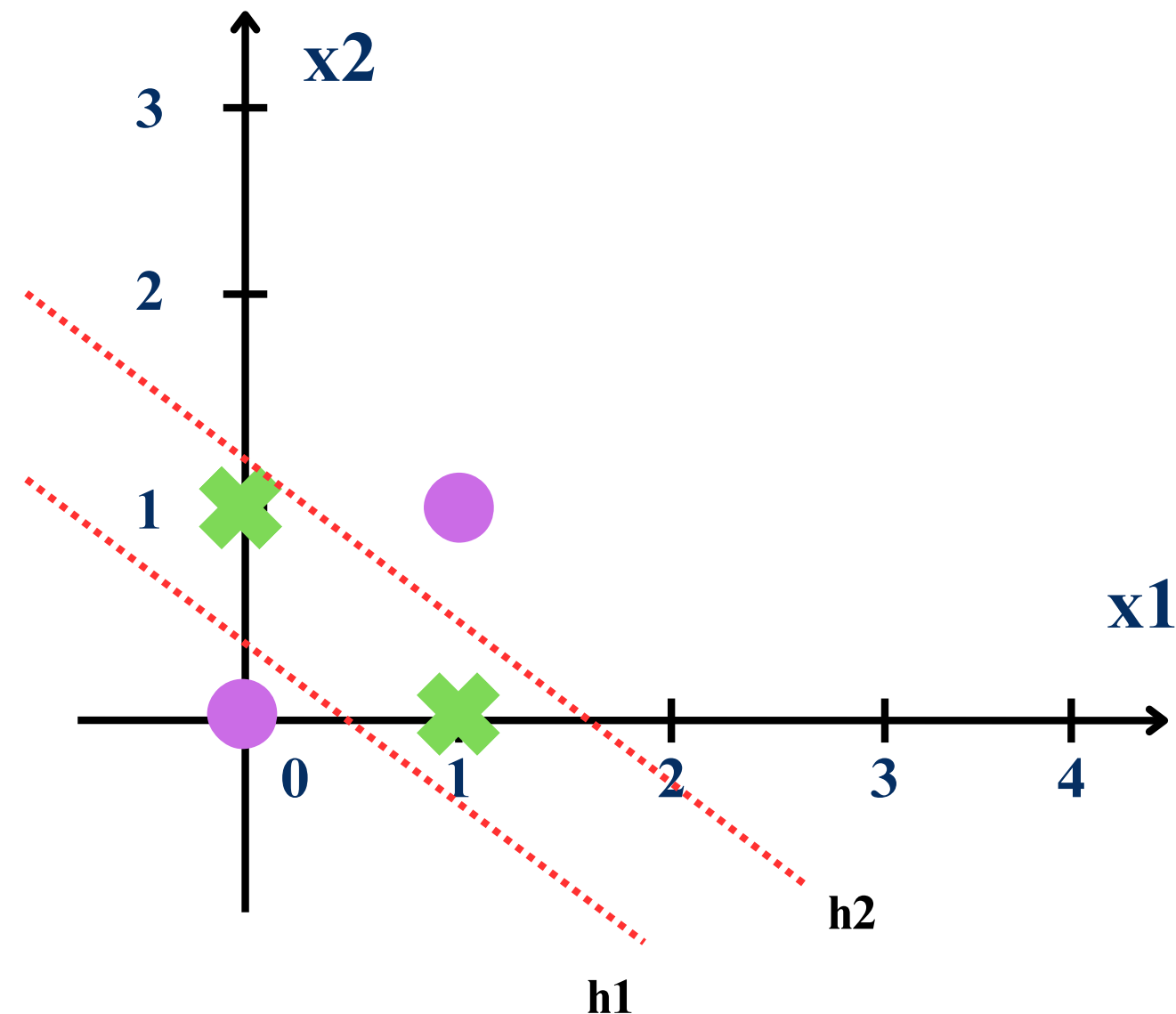
- Wagi inputów w_n
- Wartość biasu b
- Funkcje aktywacji $f(\sum)$

$w_1 = 1$
 $w_2 = -2$
 $b = -0.5$

Sieci typu MLP

Schemat pracy sieci

$$x_1, x_2 \rightarrow h_1, h_2 \rightarrow y \rightarrow \text{wynik}$$



h1 neuron

Zawiera:

- Wagi inputów w_n
- Wartość biasu b
- Funkcje aktywacji $f(\sum)$

$w_1 = 1$
 $w_2 = 1$
 $b = -0.5$

Czy cokolwiek się aktywowało? (**OR**)

$$y = -x + 0.5$$

h2 neuron

Zawiera:

- Wagi inputów w_n
- Wartość biasu b
- Funkcje aktywacji $f(\sum)$

$w_1 = 1$
 $w_2 = 1$
 $b = -1.5$

Czy wszystko się aktywowało? (**AND**)

$$y = -x + 1.5$$



y neuron

Zawiera:

- Wagi inputów w_n
- Wartość biasu b
- Funkcje aktywacji $f(\sum)$

$w_1 = 1$
 $w_2 = -2$
 $b = -0.5$

Czy coś się aktywowało w h1 ale nie w h2?
(**XOR**)

$$y = 0.5x - 0.25$$

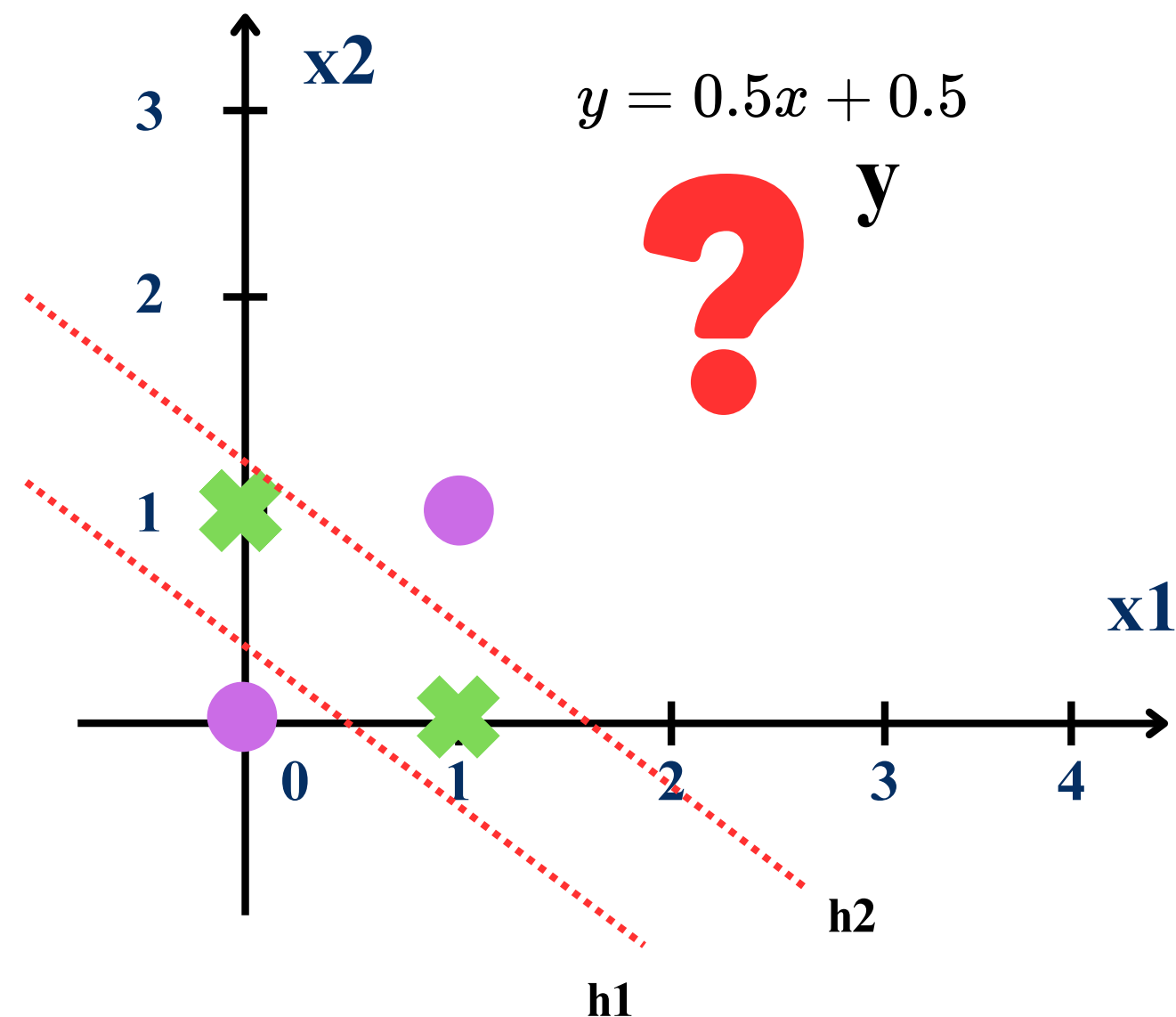
Sieci typu MLP



Wydział
Biologii

Schemat pracy sieci

$$x_1, x_2 \rightarrow h_1, h_2 \rightarrow y \rightarrow \text{wynik}$$



h1 neuron

Zawiera:

- Wagi inputów w_n
- Wartość biasu b
- Funkcje aktywacji $f(\sum)$

$w_1 = 1$
 $w_2 = 1$
 $b = -0.5$

Czy cokolwiek się aktywowało? (OR)

$$y = -x + 0.5$$

h2 neuron

Zawiera:

- Wagi inputów w_n
- Wartość biasu b
- Funkcje aktywacji $f(\sum)$

$w_1 = 1$
 $w_2 = 1$
 $b = -1.5$

Czy wszystko się aktywowało? (AND)

$$y = -x + 1.5$$



y neuron

Zawiera:

- Wagi inputów w_n
- Wartość biasu b
- Funkcje aktywacji $f(\sum)$

$w_1 = 1$
 $w_2 = -2$
 $b = -0.5$

Czy coś się aktywowało w h1 ale nie w h2?
(XOR)

$$y = 0.5x - 0.25$$

Sieci typu MLP

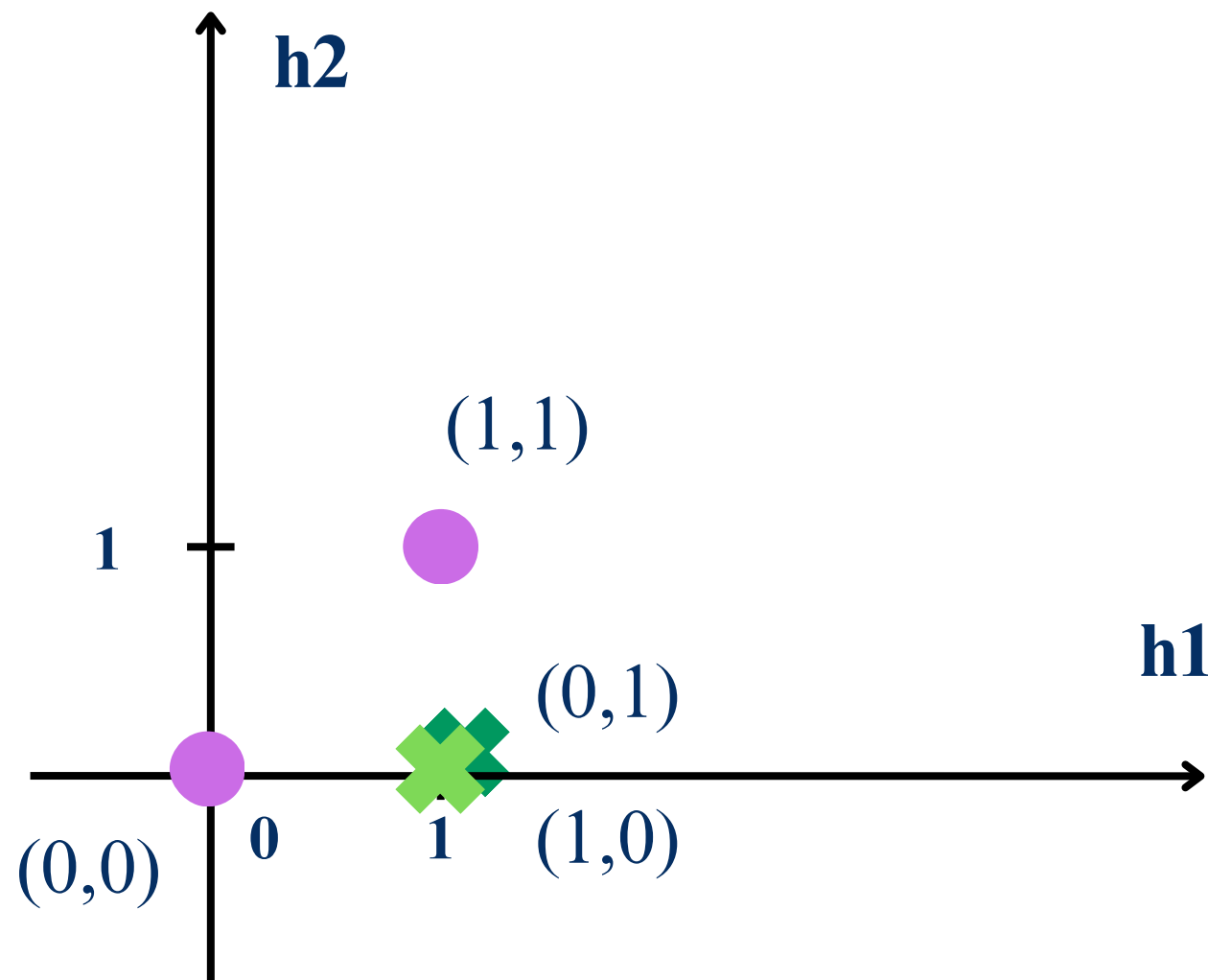


Wydział
Biologii

Schemat pracy sieci

$$x_1, x_2 \rightarrow h_1, h_2 \rightarrow y \rightarrow \text{wynik}$$

Nowa przestrzeń!!



h1 neuron

Zawiera:

- Wagi inputów w_n
- Wartość biasu b
- Funkcje aktywacji $f(\sum)$

$$w_1 = 1$$
$$w_2 = 1$$
$$b = -0.5$$

Czy cokolwiek się aktywowało? (**OR**)

$$y = -x + 0.5$$

h2 neuron

Zawiera:

- Wagi inputów w_n
- Wartość biasu b
- Funkcje aktywacji $f(\sum)$

$$w_1 = 1$$
$$w_2 = 1$$
$$b = -1.5$$

Czy wszystko się aktywowało? (**AND**)

$$y = -x + 1.5$$



y neuron

Zawiera:

- Wagi inputów w_n
- Wartość biasu b
- Funkcje aktywacji $f(\sum)$

$$w_1 = 1$$
$$w_2 = -2$$
$$b = -0.5$$

Czy coś się aktywowało w h1 ale nie w h2?
(**XOR**)

$$y = 0.5x - 0.25$$

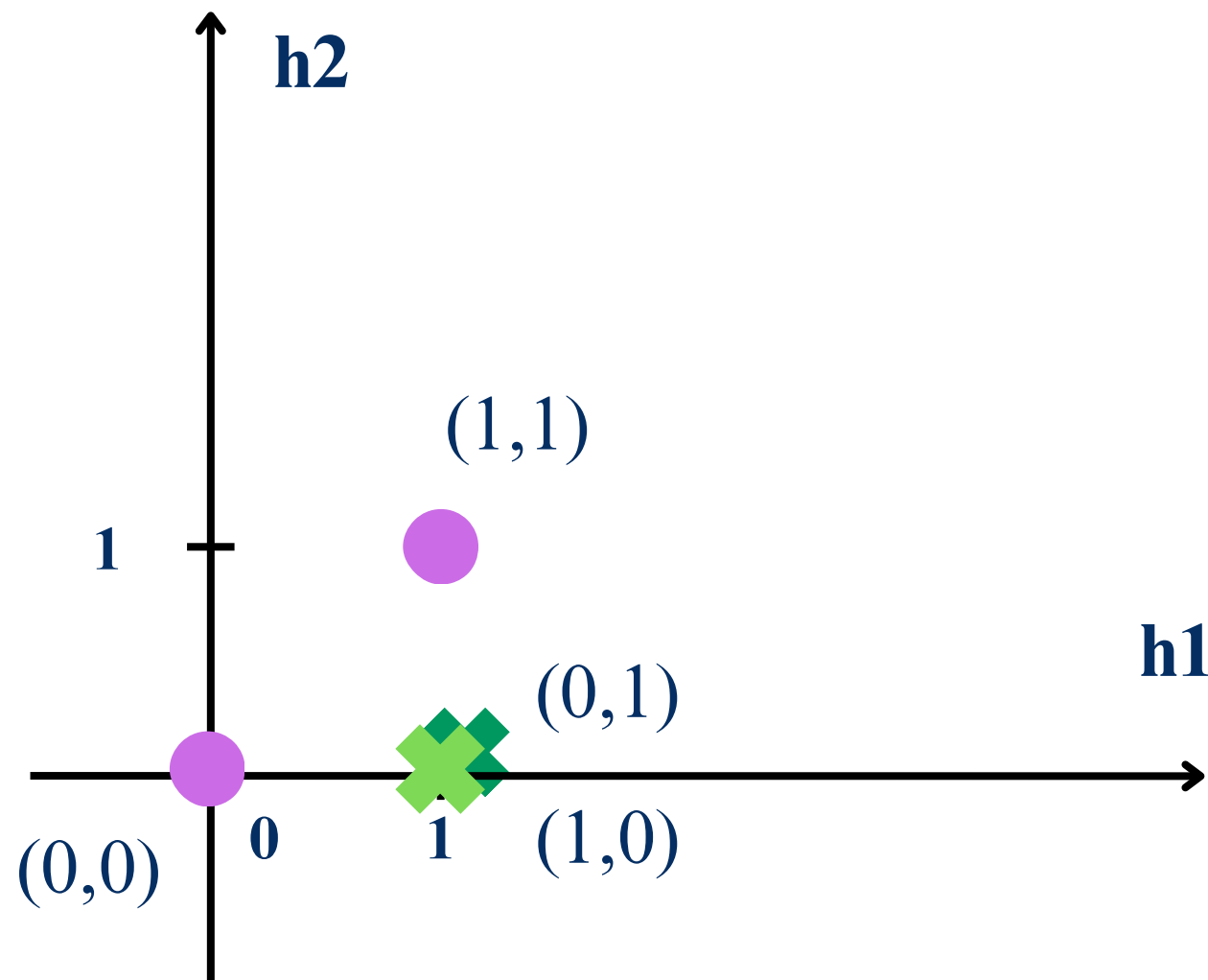
Sieci typu MLP



Wydział
Biologii

Schemat pracy sieci

$$x_1, x_2 \rightarrow h_1, h_2 \rightarrow y \rightarrow \text{wynik}$$



h1 neuron

Zawiera:	
• Wagi inputów	w_n
• Wartość biasu	b
• Funkcje aktywacji	$f(\sum)$

$$w_1 = 1$$
$$w_2 = 1$$
$$b = -0.5$$

Czy cokolwiek się aktywowało? (**OR**)

$$y = -x + 0.5$$

h2 neuron

Zawiera:	
• Wagi inputów	w_n
• Wartość biasu	b
• Funkcje aktywacji	$f(\sum)$

$$w_1 = 1$$
$$w_2 = 1$$
$$b = -1.5$$

Czy wszystko się aktywowało? (**AND**)

$$y = -x + 1.5$$



y neuron

Zawiera:	
• Wagi inputów	w_n
• Wartość biasu	b
• Funkcje aktywacji	$f(\sum)$

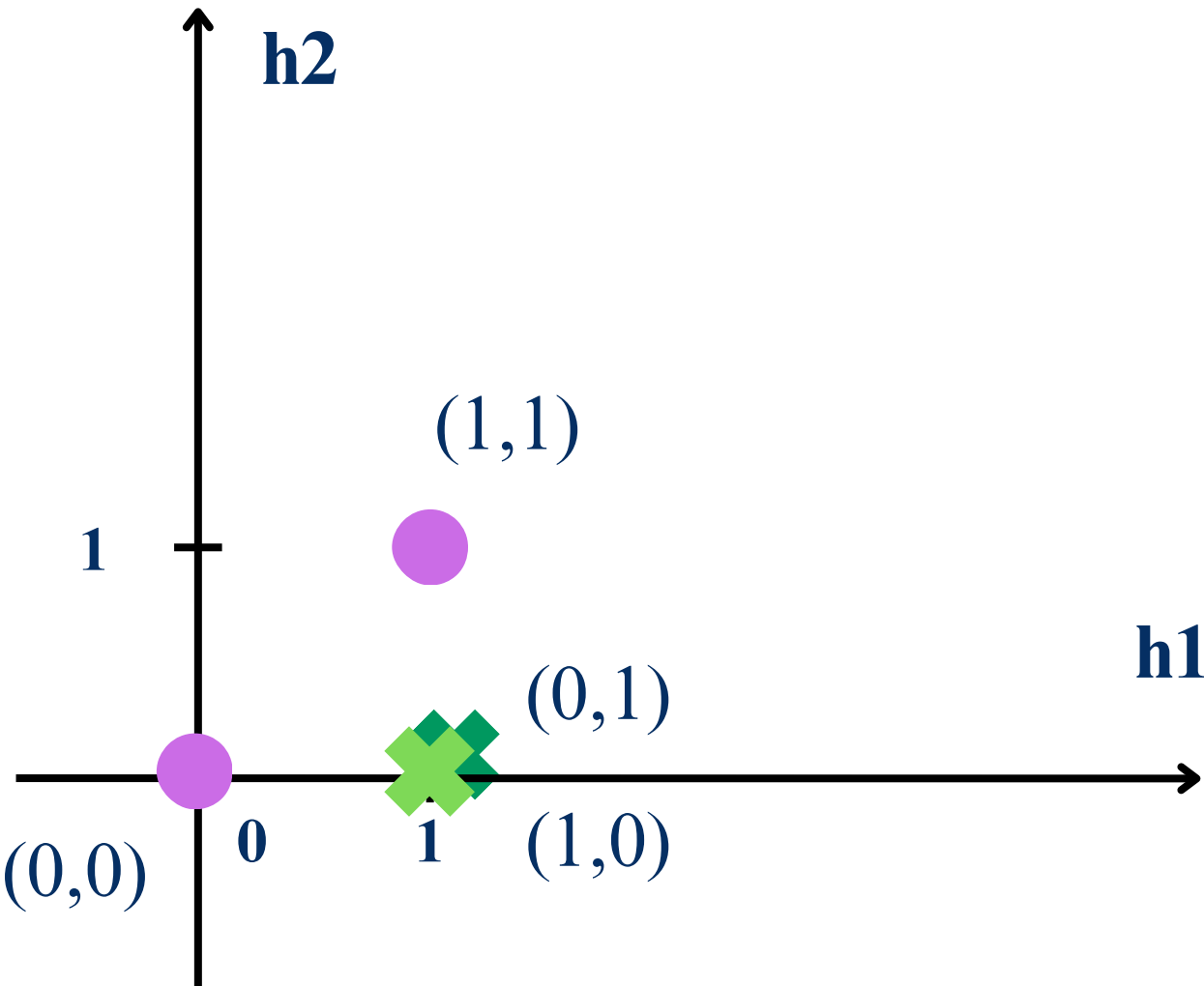
$$w_1 = 1$$
$$w_2 = -2$$
$$b = -0.5$$

Czy coś się aktywowało w h1 ale nie w h2?
(**XOR**)

$$y = 0.5x - 0.25$$

Schemat pracy sieci

$$x_1, x_2 \rightarrow h_1, h_2 \rightarrow y \rightarrow \text{wynik}$$



x1	x2	suma (x1+x2)	H1 (próg 0.5)	H2 (próg 1.5)	→ miejsce w H1,H2
0	0	0	0	0	(0,0)
1	0	1	1	0	(1,0)
0	1	1	1	0	(1,0)
1	1	2	1	1	(1,1)

y neuron

Zawiera:

- Wagi inputów
- Wartość biasu
- Funkcje aktywacji

w_n
 b
 $f(\sum)$

$$\begin{aligned}w_1 &= 1 \\w_2 &= -2 \\b &= -0.5\end{aligned}$$

Czy coś się aktywowało w h1 ale nie w h2?
(XOR)

$$y = 0.5x - 0.25$$

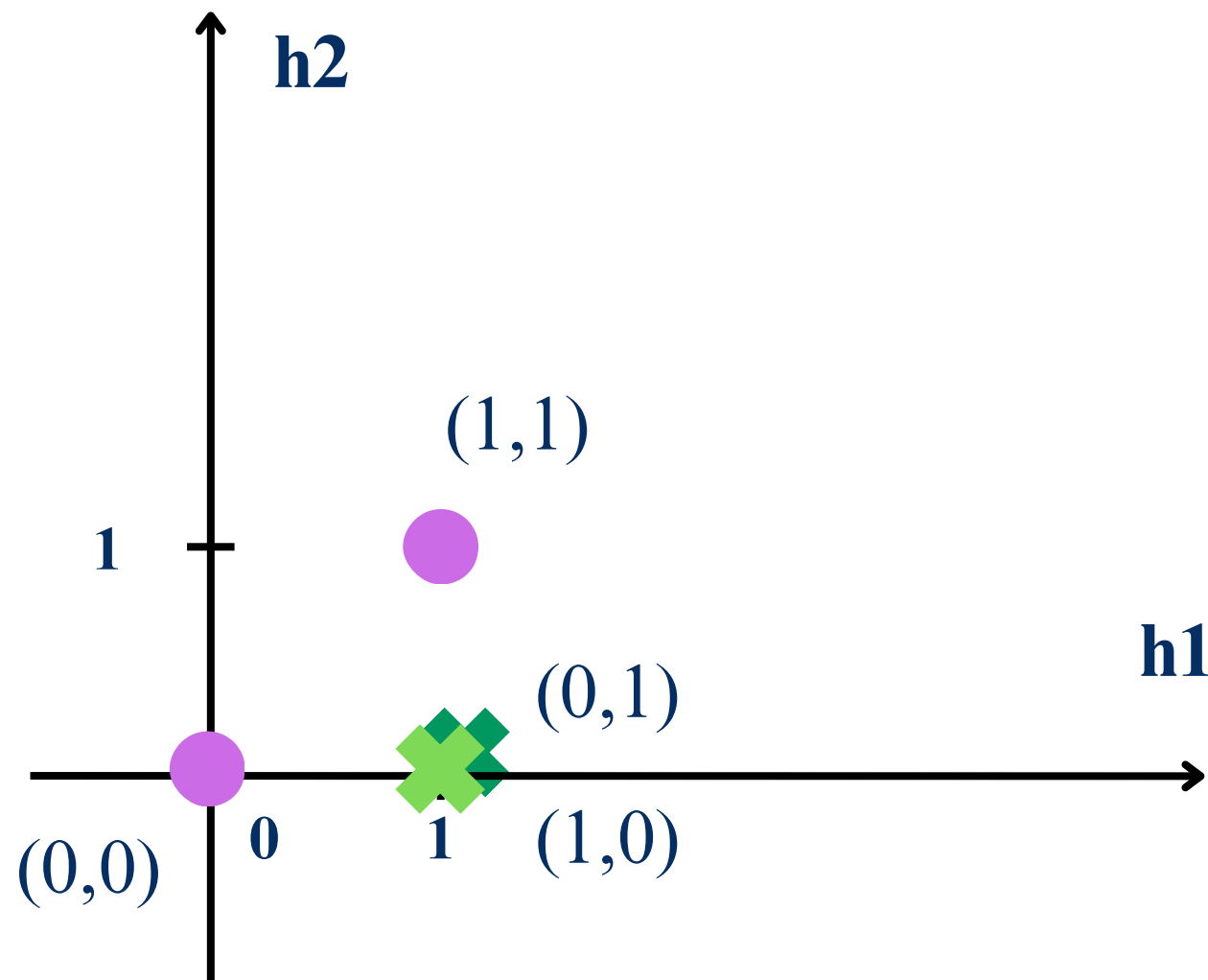
Sieci typu MLP



Wydział
Biologii

Schemat pracy sieci

$$x_1, x_2 \rightarrow h_1, h_2 \rightarrow y \rightarrow \text{wynik}$$



h1 neuron

Zawiera:	
• Wagi inputów	w_n
• Wartość biasu	b
• Funkcje aktywacji	$f(\sum)$

$$w_1 = 1$$
$$w_2 = 1$$
$$b = -0.5$$

Czy cokolwiek się aktywowało? (**OR**)

$$y = -x + 0.5$$

h2 neuron

Zawiera:	
• Wagi inputów	w_n
• Wartość biasu	b
• Funkcje aktywacji	$f(\sum)$

$$w_1 = 1$$
$$w_2 = 1$$
$$b = -1.5$$

Czy wszystko się aktywowało? (**AND**)

$$y = -x + 1.5$$



y neuron

Zawiera:	
• Wagi inputów	w_n
• Wartość biasu	b
• Funkcje aktywacji	$f(\sum)$

$$w_1 = 1$$
$$w_2 = -2$$
$$b = -0.5$$

Czy coś się aktywowało w h1 ale nie w h2?
(**XOR**)

$$y = 0.5x - 0.25$$

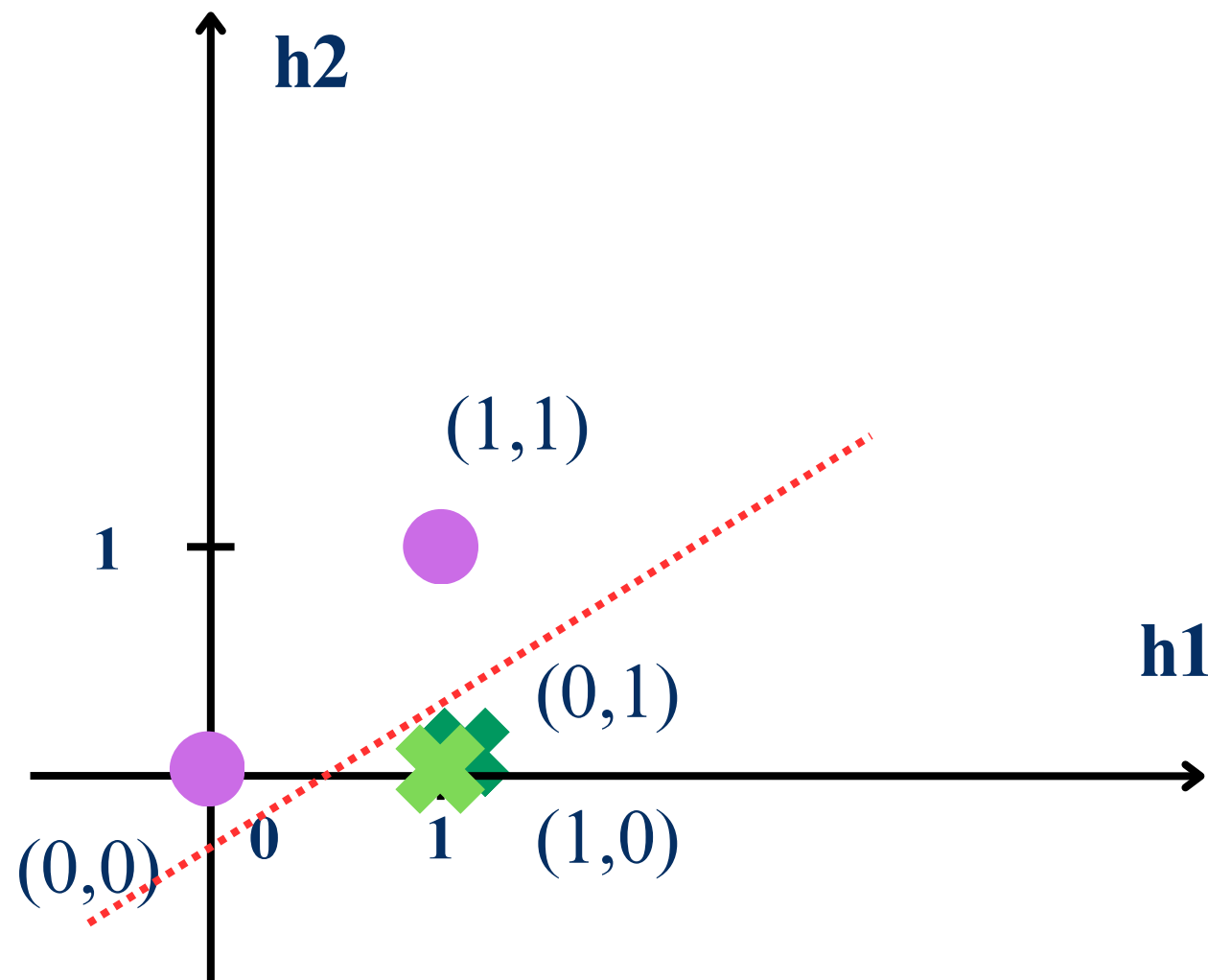
Sieci typu MLP



Wydział
Biologii

Schemat pracy sieci

$$x_1, x_2 \rightarrow h_1, h_2 \rightarrow y \rightarrow \text{wynik}$$



h1 neuron

Zawiera:

- Wagi inputów w_n
- Wartość biasu b
- Funkcje aktywacji $f(\sum)$

$w_1 = 1$
 $w_2 = 1$
 $b = -0.5$

Czy cokolwiek się aktywowało? (**OR**)

$$y = -x + 0.5$$

h2 neuron

Zawiera:

- Wagi inputów w_n
- Wartość biasu b
- Funkcje aktywacji $f(\sum)$

$w_1 = 1$
 $w_2 = 1$
 $b = -1.5$

Czy wszystko się aktywowało? (**AND**)

$$y = -x + 1.5$$



y neuron

Zawiera:

- Wagi inputów w_n
- Wartość biasu b
- Funkcje aktywacji $f(\sum)$

$w_1 = 1$
 $w_2 = -2$
 $b = -0.5$

Czy coś się aktywowało w h1 ale nie w h2?
(**XOR**)

$$y = 0.5x - 0.25$$

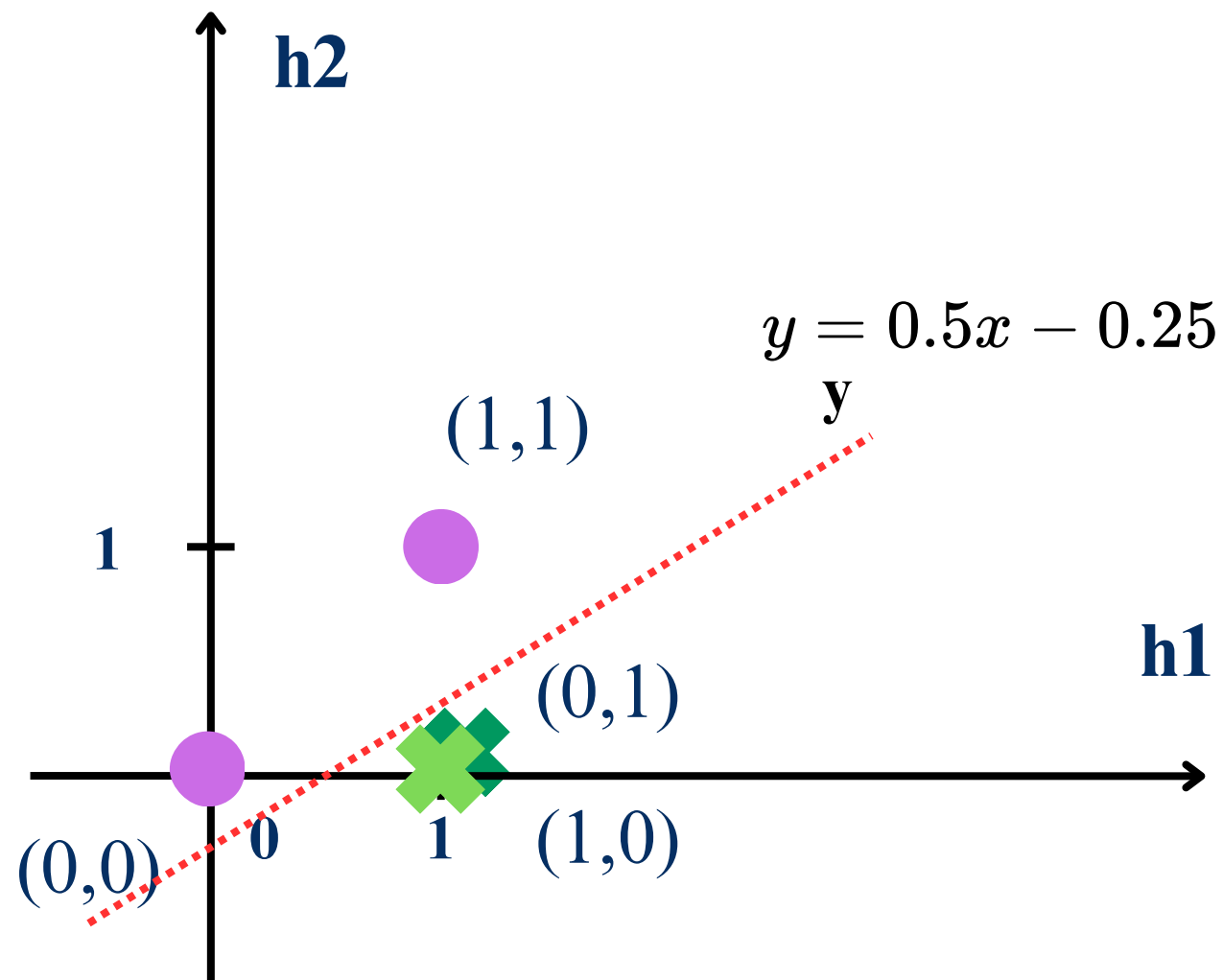
Sieci typu MLP



Wydział
Biologii

Schemat pracy sieci

$$x_1, x_2 \rightarrow h_1, h_2 \rightarrow y \rightarrow \text{wynik}$$



h1 neuron

Zawiera:	
• Wagi inputów	w_n
• Wartość biasu	b
• Funkcje aktywacji	$f(\sum)$

$$w_1 = 1$$
$$w_2 = 1$$
$$b = -0.5$$

Czy cokolwiek się aktywowało? (OR)

$$y = -x + 0.5$$

h2 neuron

Zawiera:	
• Wagi inputów	w_n
• Wartość biasu	b
• Funkcje aktywacji	$f(\sum)$

$$w_1 = 1$$
$$w_2 = 1$$
$$b = -1.5$$

Czy wszystko się aktywowało? (AND)

$$y = -x + 1.5$$



y neuron

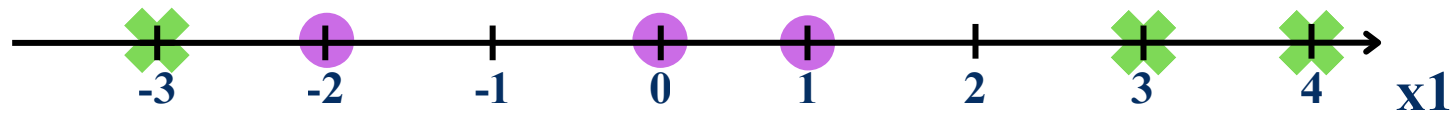
Zawiera:	
• Wagi inputów	w_n
• Wartość biasu	b
• Funkcje aktywacji	$f(\sum)$

$$w_1 = 1$$
$$w_2 = -2$$
$$b = -0.5$$

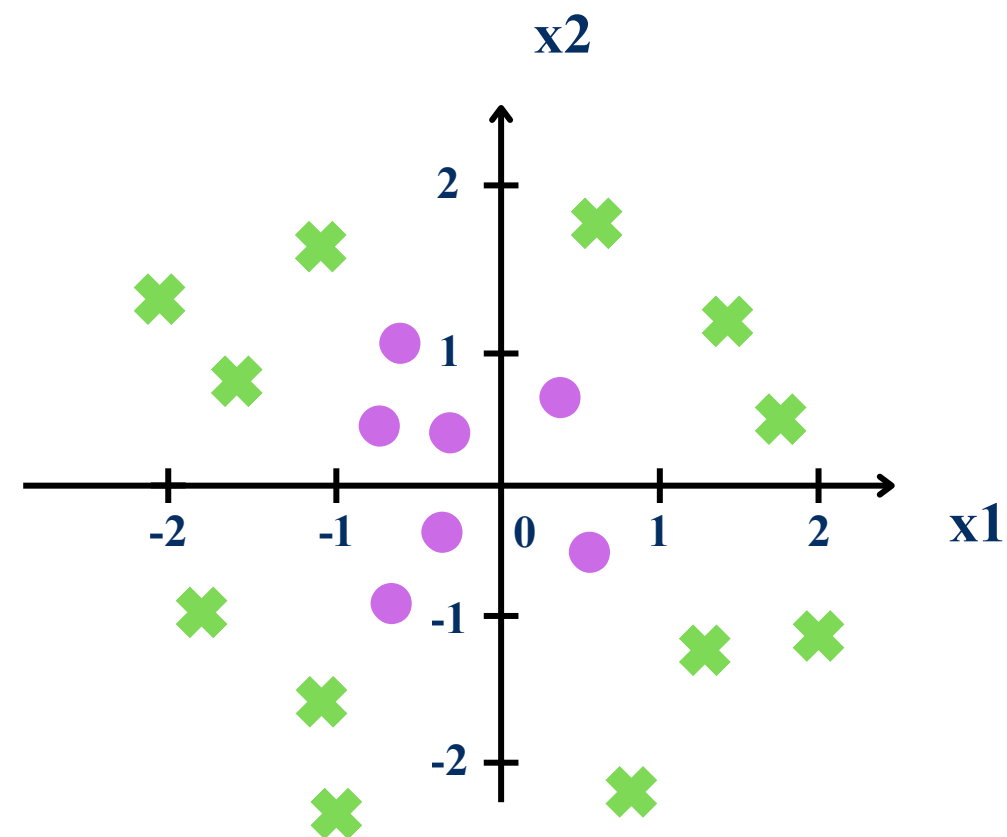
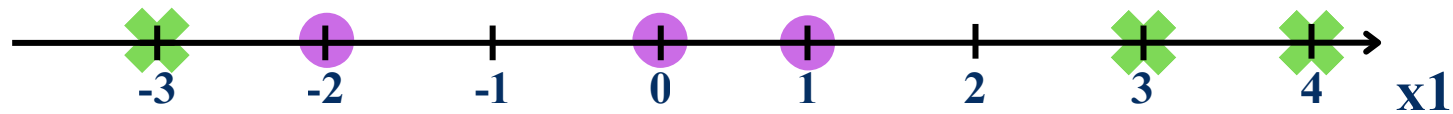
Czy coś się aktywowało w h1 ale nie w h2?
(XOR)

$$y = 0.5x - 0.25$$

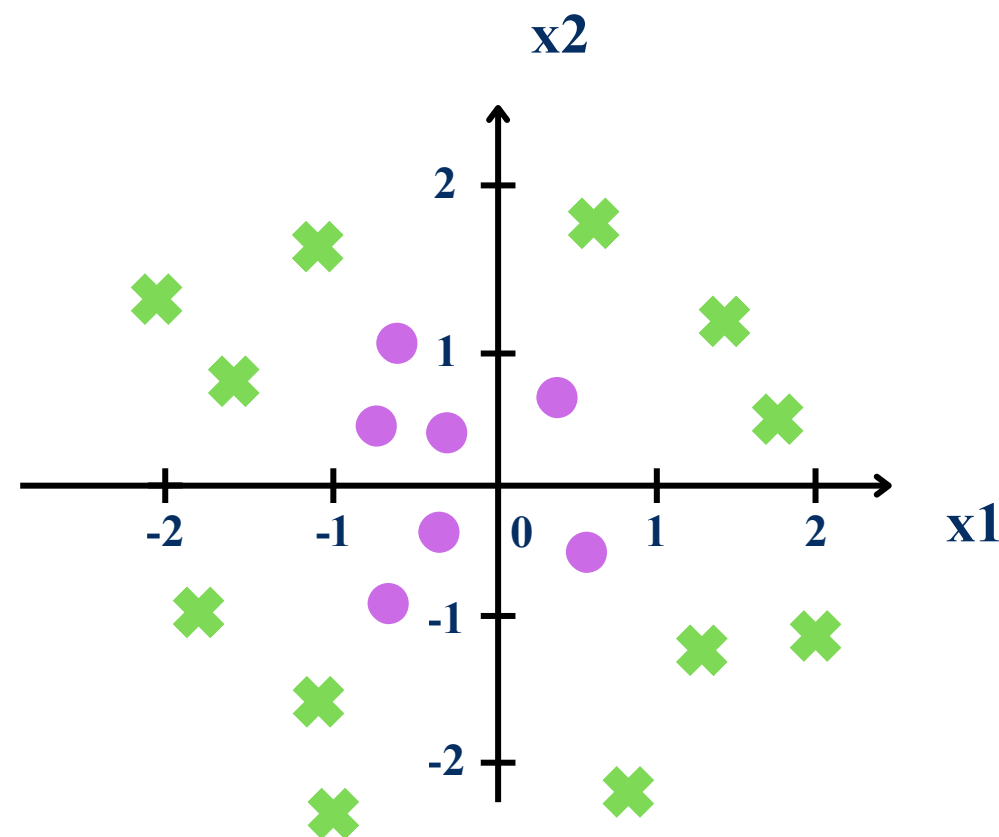
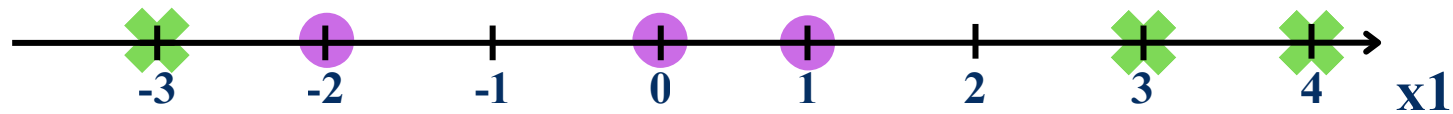
Czy można liniowo odseparować te 2 klasy?



Czy można liniowo odseparować te 2 klasy?



Czy można liniowo odseparować te 2 klasy?



Transformacja przestrzeni cech (np. przez dodanie nowych wymiarów lub nieliniowe przekształcenia) może przekształcić problem nieliniowo separowalny w liniowo separowalny.

W nowej przestrzeni dane można rozdzielić hiperpłaszczyzną, co wykorzystują m.in. sieci neuronowe oraz Support Vector Machine (Kernel trick).

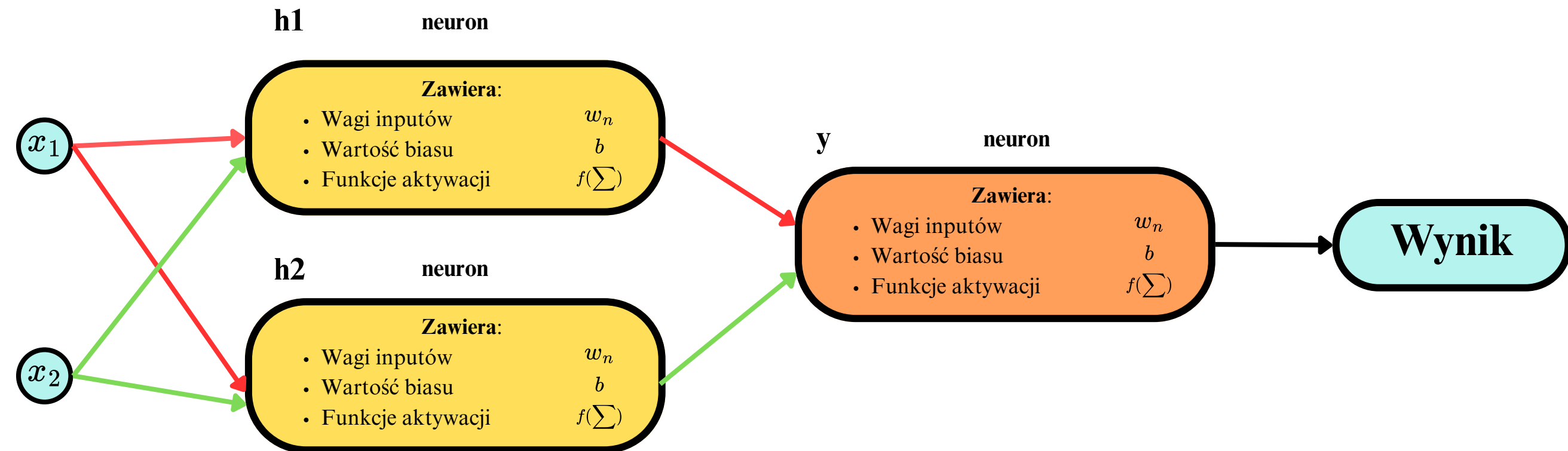
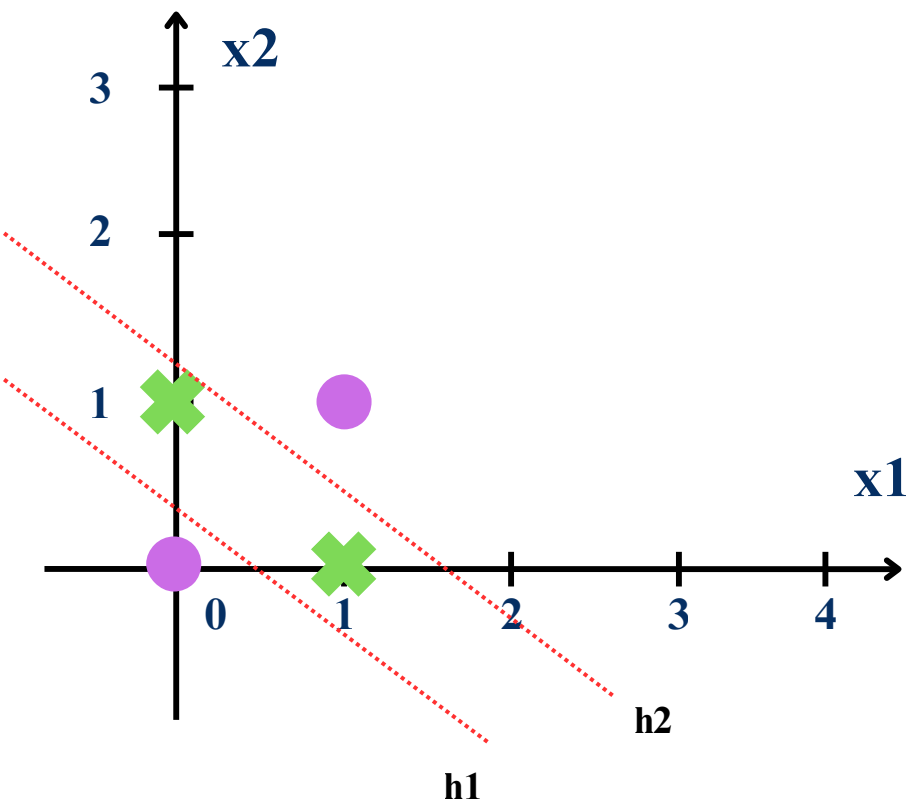
Więcej warstw - jaki jest cel?

input

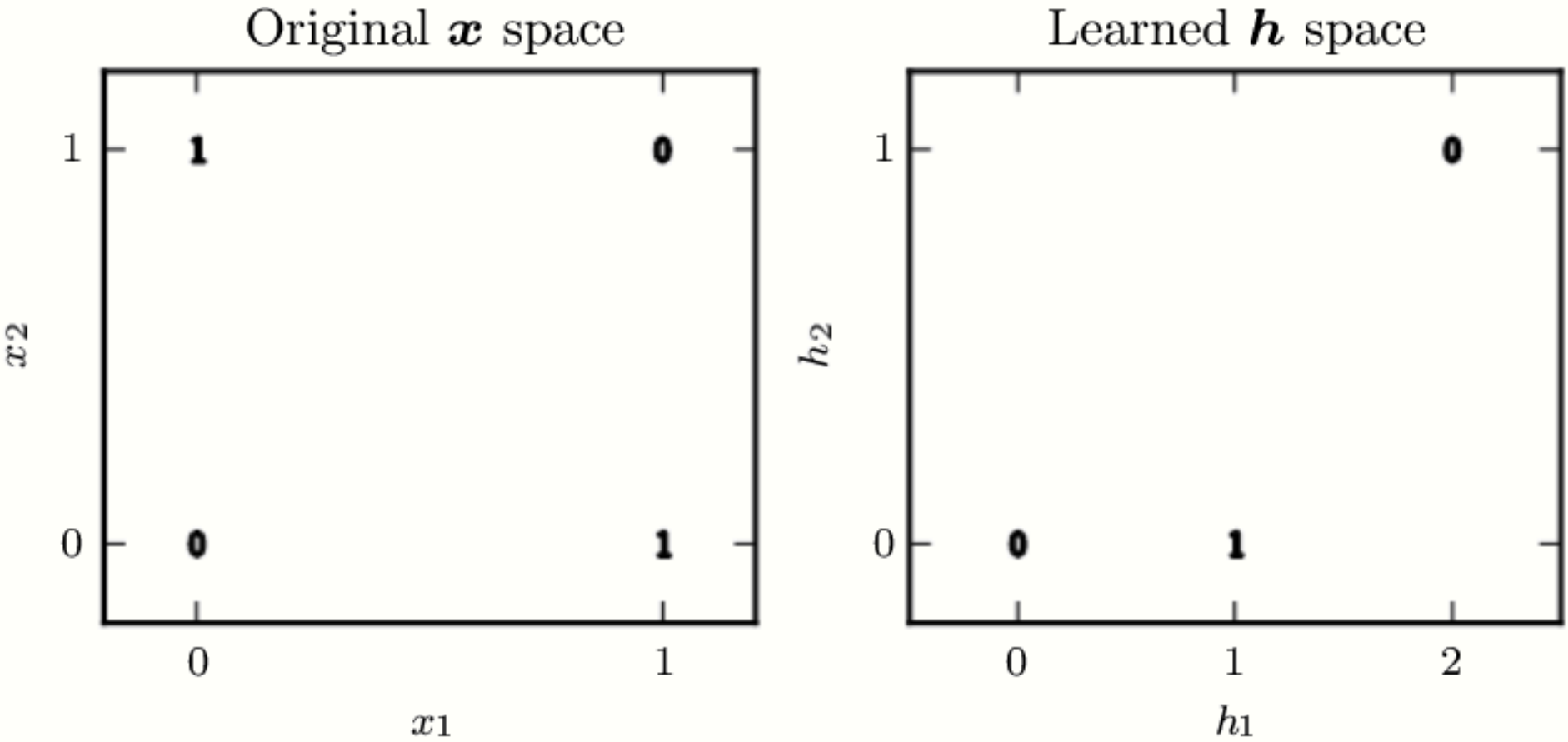
Warstwa ukryta

Neuron wyjściowy

output



Więcej warstw - jaki jest cel?

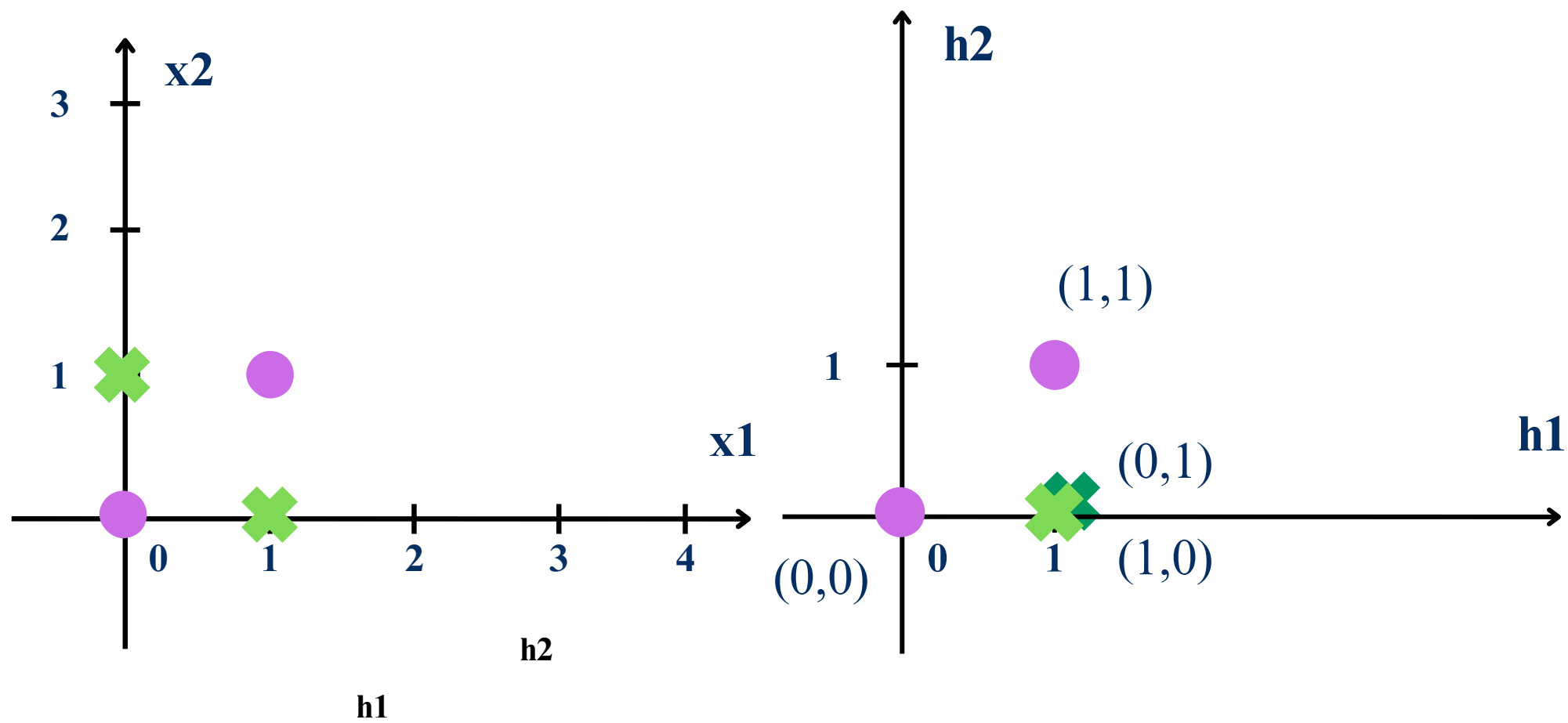


x_1	x_2	suma (x_1+x_2)	H1 (próg 0.5)	H2 (próg 1.5)	→ miejsce w H1,H2
0	0	0	0	0	(0,0)
1	0	1	1	0	(1,0)
0	1	1	1	0	(1,0)
1	1	2	1	1	(1,1)

Więcej warstw - jaki jest cel?



Wydział
Biologii

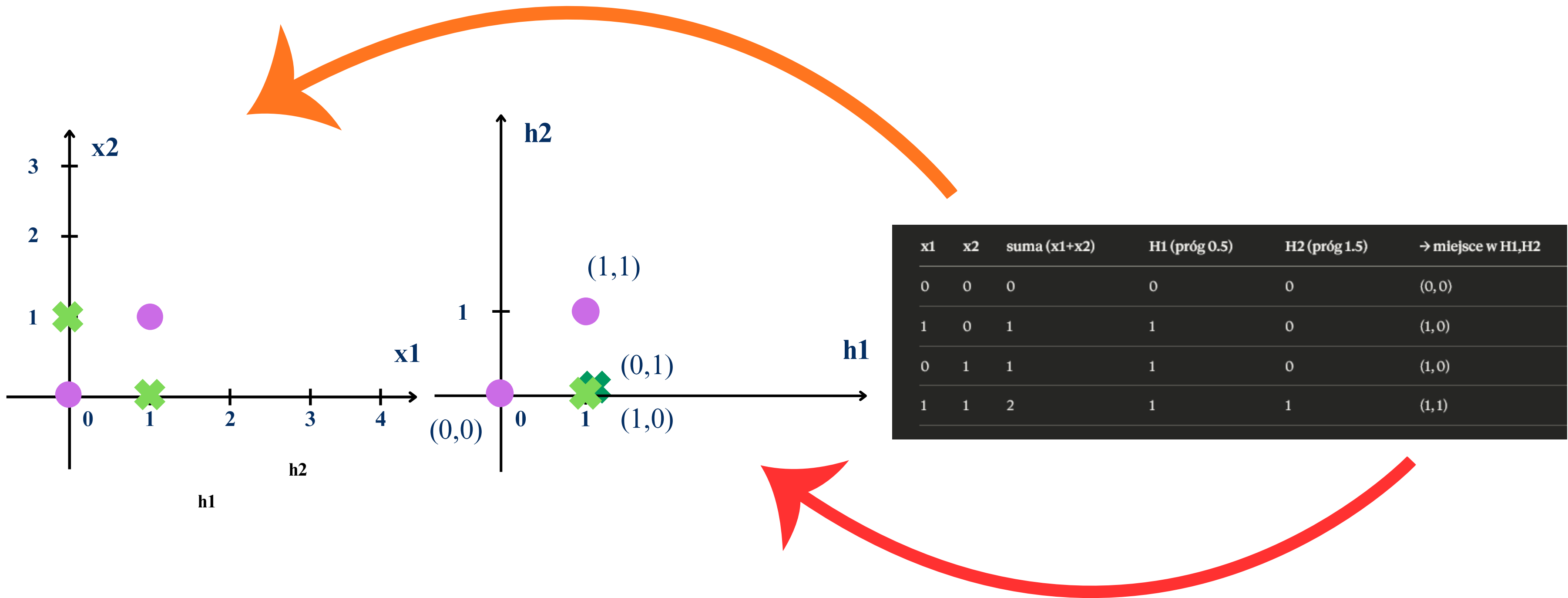


x1	x2	suma (x1+x2)	H1 (próg 0.5)	H2 (próg 1.5)	→ miejsce w H1,H2
0	0	0	0	0	(0,0)
1	0	1	1	0	(1,0)
0	1	1	1	0	(1,0)
1	1	2	1	1	(1,1)

Więcej warstw - jaki jest cel?



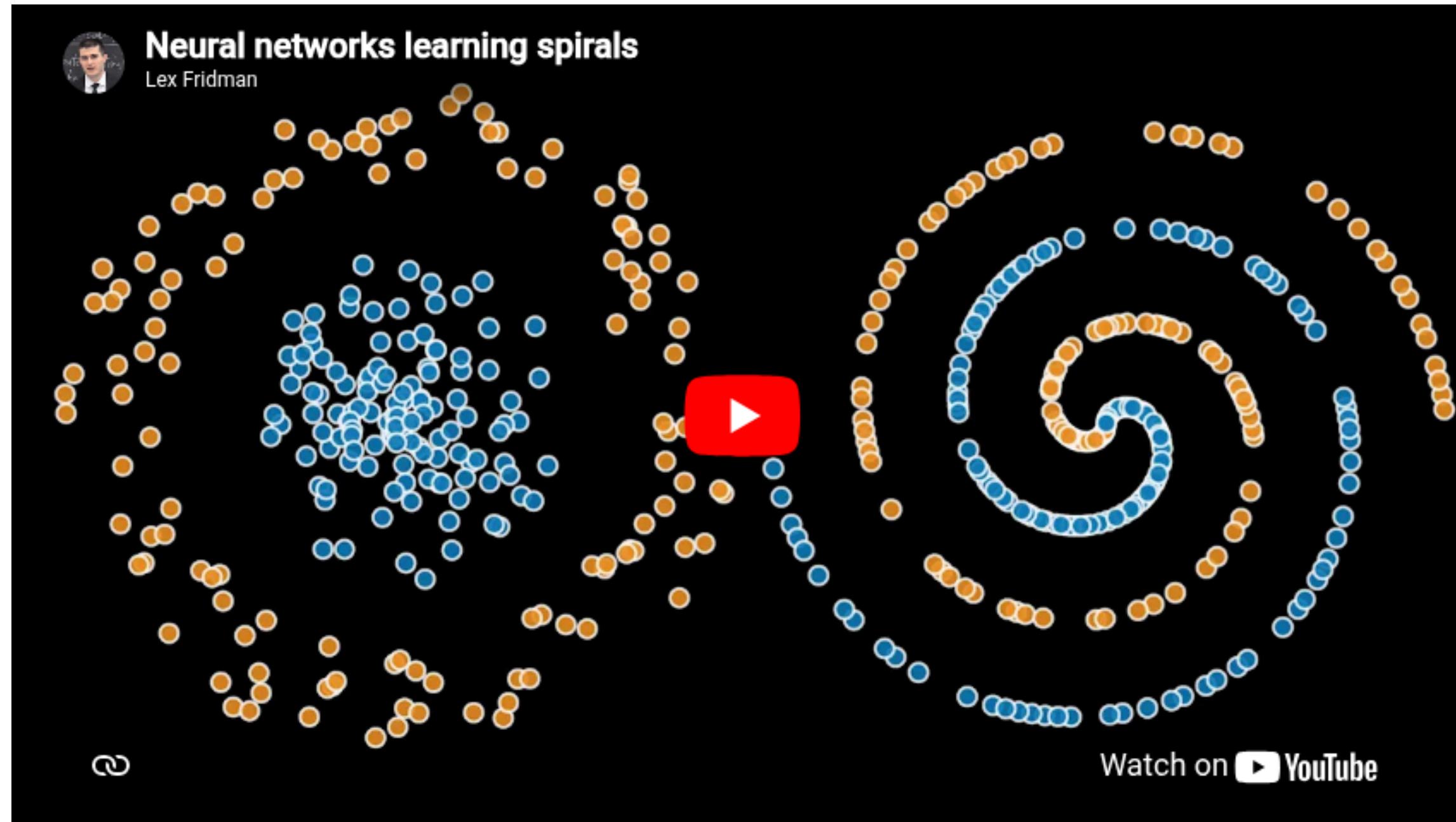
Wydział
Biologii



Więcej warstw - jaki jest cel?



Wydział
Biologii

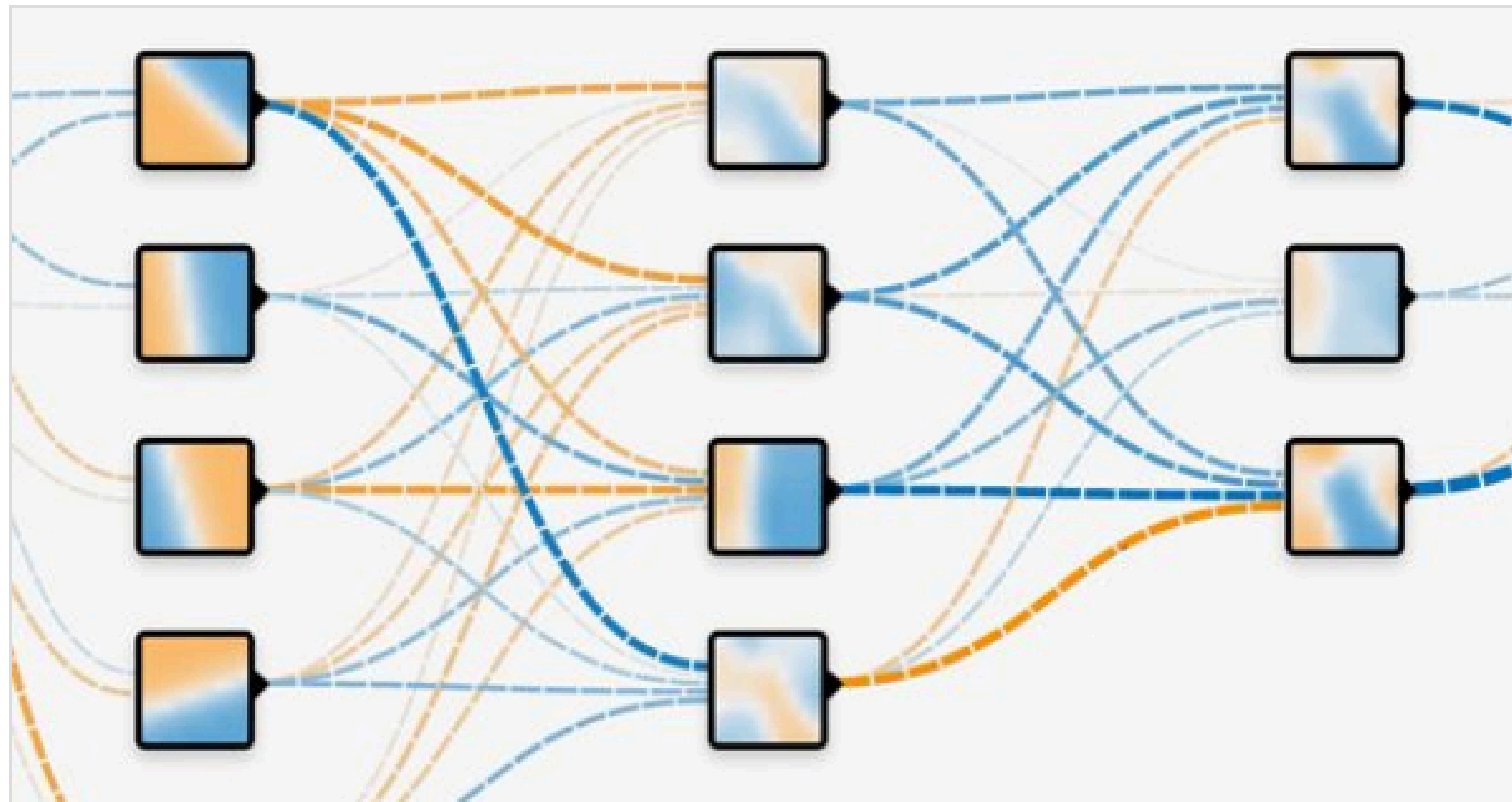


<https://www.youtube.com/watch?v=i3ZnDRrmFjg&t=48s>

Więcej warstw - jaki jest cel?



Wydział
Biologii



Tensorflow — Neural Network Playground

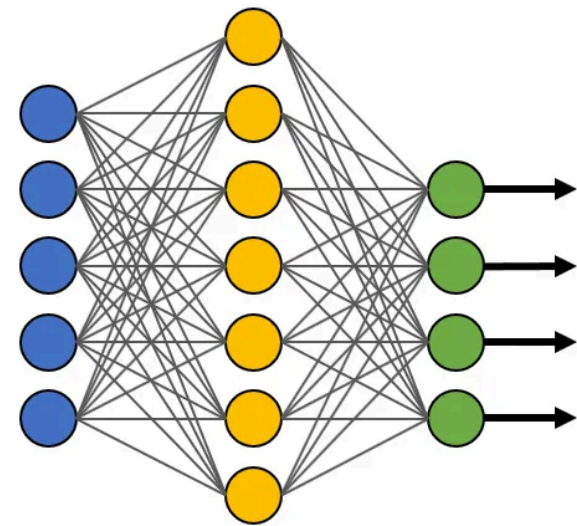
Tinker with a real neural network right here in your browser.

 [tensorflow.org /](https://www.tensorflow.org/)

<https://playground.tensorflow.org/#activation=tanh&batchSize=10&dataset=circle®Dataset=reg-plane&learningRate=0.03®ularizationRate=0&noise=0&networkShape=4,2&seed=0.77810&showTestData=false&discretize=false&percTrainData=50&x=true&y=true&xTimesY=false&xSquared=false&ySquared=false&cosX=false&sinX=false&cosY=false&sinY=false&collectStats=false&problem=classification&initZero=false&hideText=false>

- Neurony - hiperpłaszczyzny decyzyjne
- Warstwy - transformacja przestrzeni wyników

Simple Neural Network

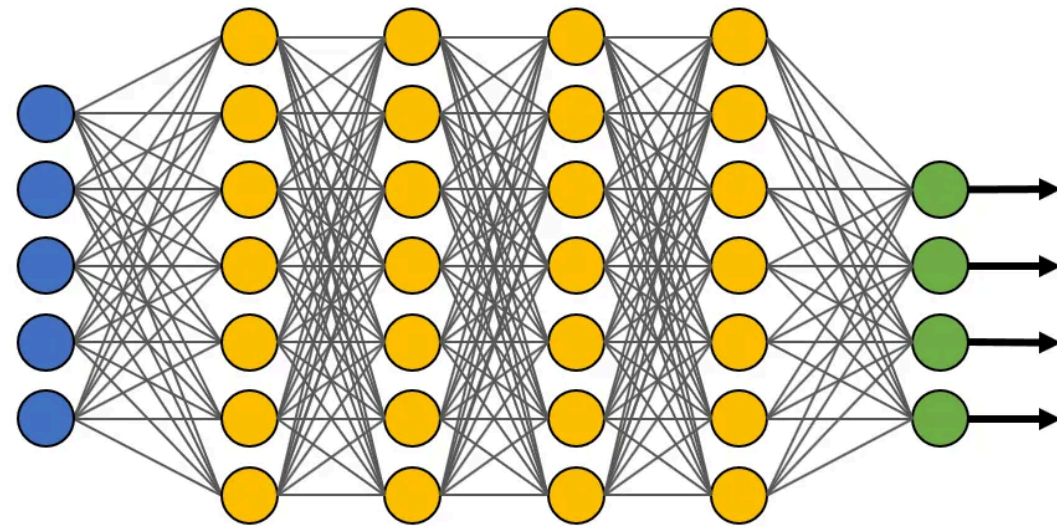


● Input Layer

● Hidden Layer

● Output Layer

Deep Learning Neural Network



Źródło: Nemade, G., A beginner's guide to neural networks and deep learning, Medium, 28.02.2025, dostęp: 14.04.2026, <https://medium.com/@nemagan/a-beginners-guide-to-neural-networks-and-deep-learning-6b5509526dd9>

Zalety wielu warstw ukrytych

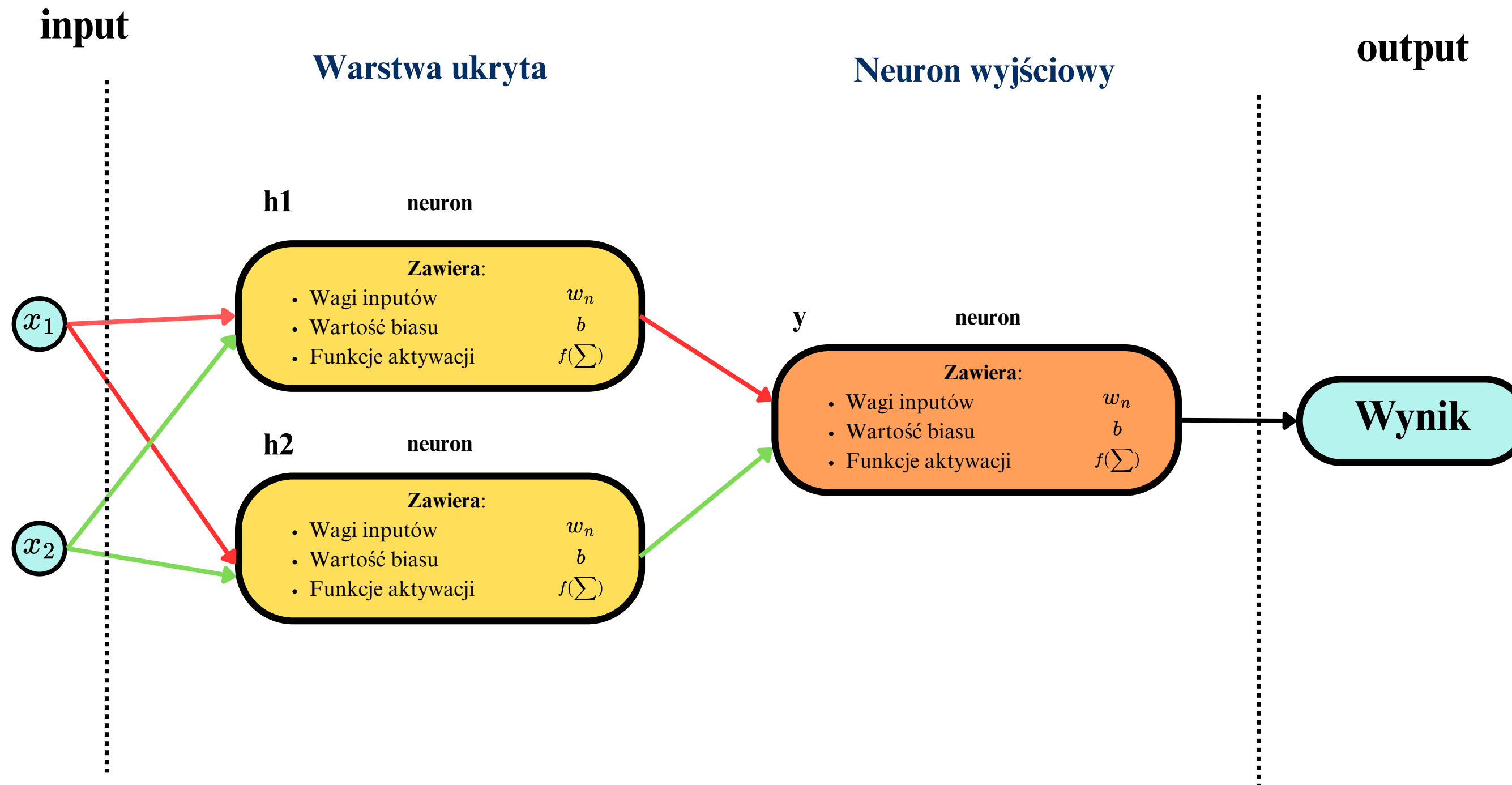
- Wysoka zdolność reprezentacji – model uczy się złożonych, nieliniowych zależności
- Hierarchiczne cechy – warstwy budują coraz bardziej abstrakcyjne reprezentacje
- Lepsze wyniki w trudnych zadaniach – np. wizja, NLP, rozpoznawanie mowy

Wady wielu warstw ukrytych

- Trudności w treningu – m.in. zanikający/eksplodujący gradient i skomplikowana optymalizacja
- Duże koszty obliczeniowe – więcej parametrów = większe zużycie czasu i pamięci
- Ryzyko przeuczenia i słaba interpretowalność – łatwiej o overfitting i trudniej zrozumieć model

Założmy, że przeprowadziliśmy **test wiedzy biologicznej i matematycznej** wśród studentów 5 roku **biologii, bioinformatyki** oraz **informatyki**. W oparciu o te dane, chcemy stworzyć sieć neuronową zdolną do rozróżniania pomiędzy studentami tych kierunków na podstawie jedynie wyników tych dwóch testów.

Zadania klasyfikacji wieloklasowej



Architektura naszej starej sieci
do rozwiązywania XOR

Założmy, że przeprowadziliśmy **test wiedzy biologicznej i matematycznej** wśród studentów 5 roku **biologii, bioinformatyki** oraz **informatyki**. W oparciu o te dane, chcemy stworzyć sieć neuronową zdolną do rozróżniania pomiędzy studentami tych kierunków na podstawie jedynie wyników tych dwóch testów.

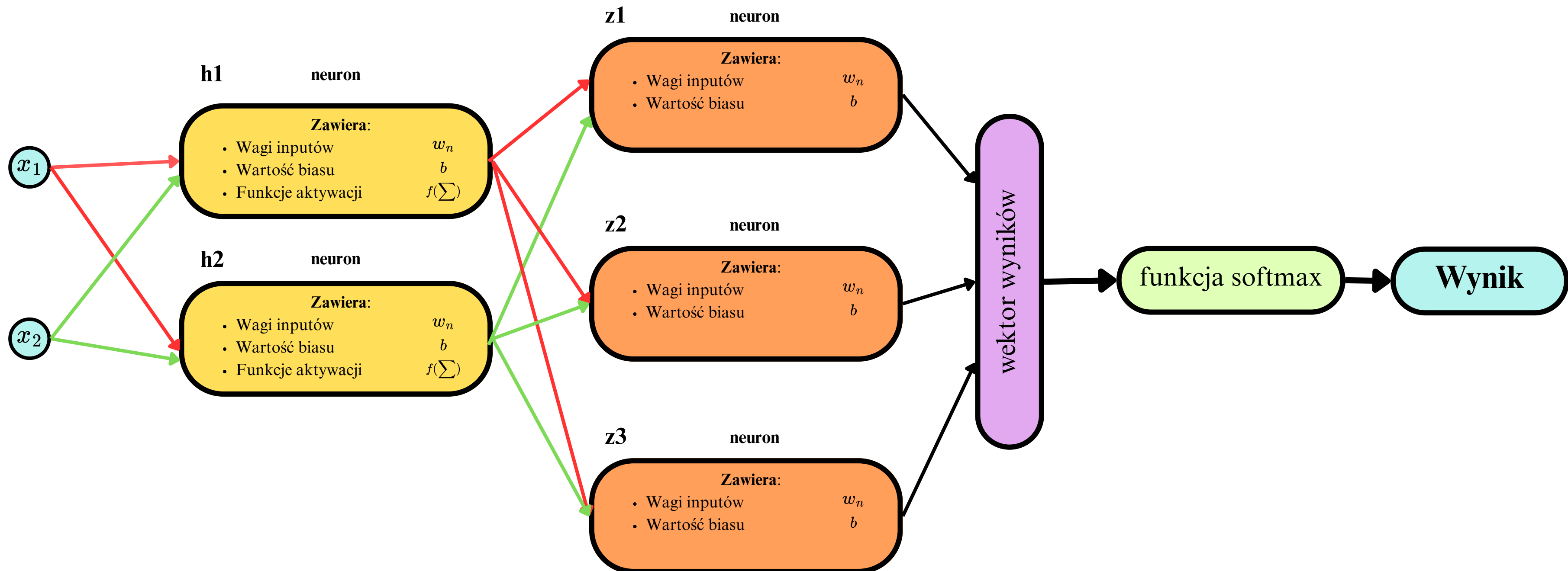
Zadania klasyfikacji wieloklasowej

input

Warstwa ukryta

Neurony warstwy wyjściowej

output



Zadania klasyfikacji wieloklasowej

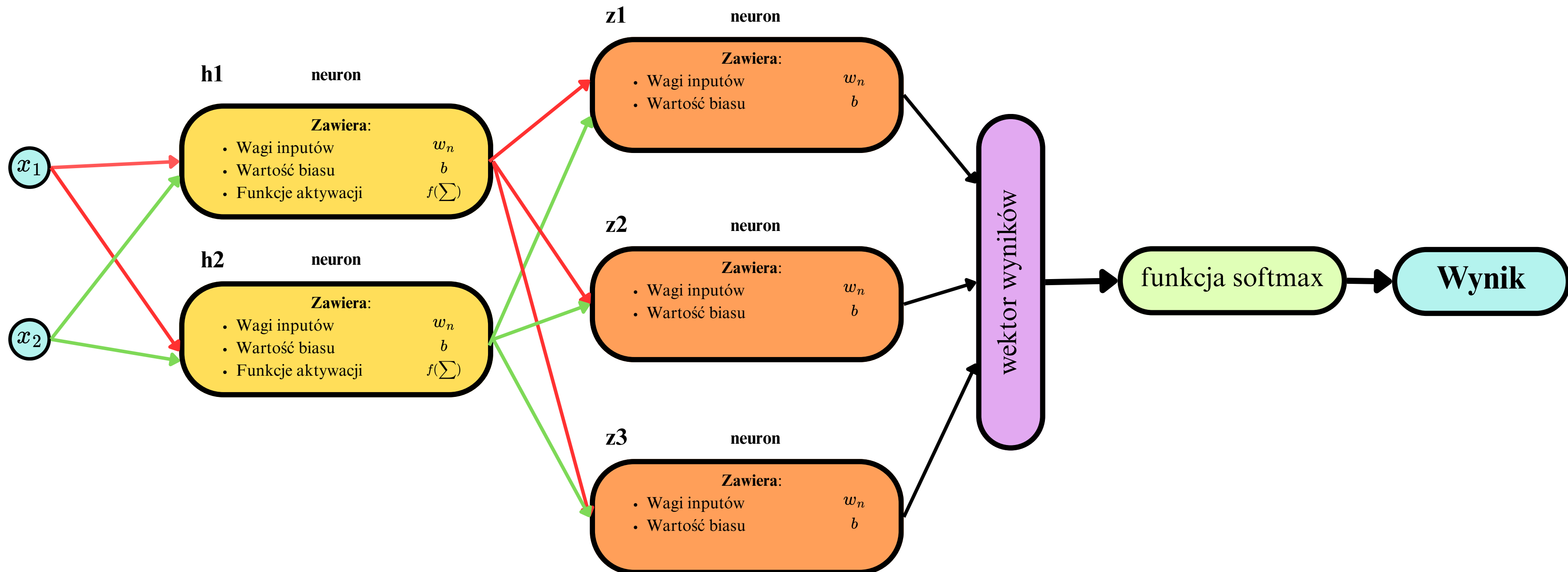
$$x_1, x_2 \rightarrow h_1, h_2 \rightarrow z_1, z_2, z_3 \rightarrow \text{softmax} \rightarrow \text{wynik}$$

input

Warstwa ukryta

Neurony warstwy wyjściowej

output



Zadania klasyfikacji wieloklasowej

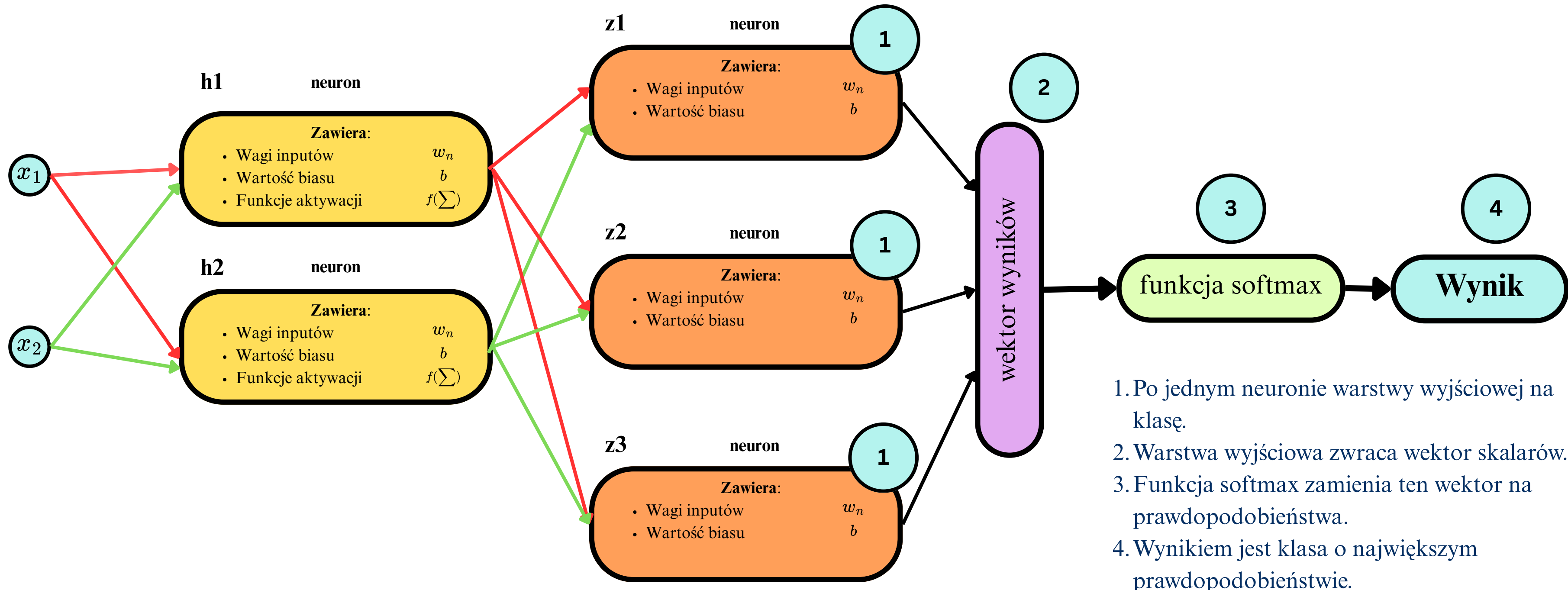
$$x_1, x_2 \rightarrow h_1, h_2 \rightarrow z_1, z_2, z_3 \rightarrow \text{softmax} \rightarrow \text{wynik}$$

input

Warstwa ukryta

Neurony warstwy wyjściowej

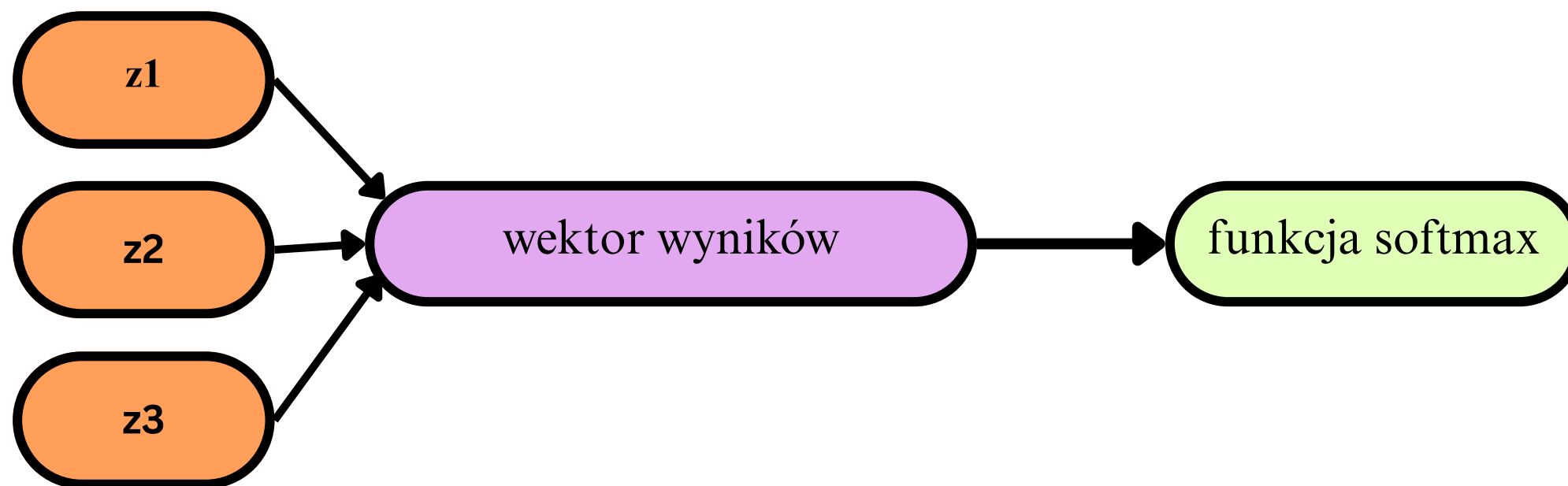
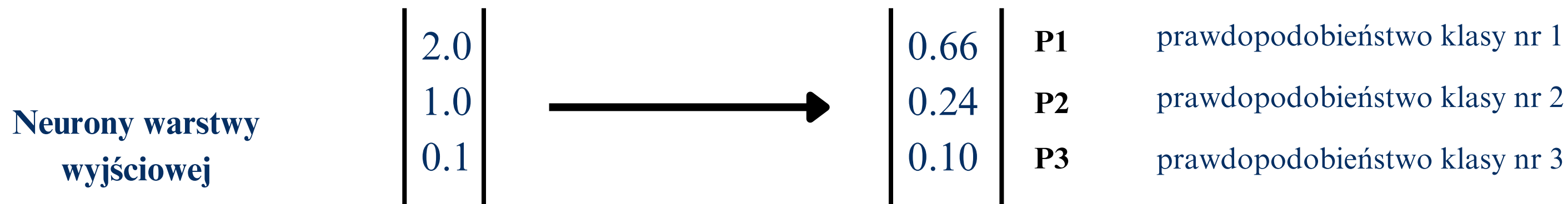
output



Funkcja softmax



Wydział
Biologii



Funkcja softmax zamienia wektor dowolnych liczb (**logitów**) na rozkład prawdopodobieństwa, gdzie wszystkie wartości są dodatnie i sumują się do 1. Wzmacnia największe wartości, a jednocześnie zmniejsza mniejsze, dzięki czemu wyraźnie wskazuje najbardziej prawdopodobną klasę. Jest najczęściej stosowana w ostatniej warstwie sieci neuronowych w zadaniach klasyfikacji wieloklasowej. **Funkcja ta w pewnym sensie zastępuje miejsce funkcji aktywacji w neuronach wyjściowych!**

Zadania klasyfikacji wieloklasowej

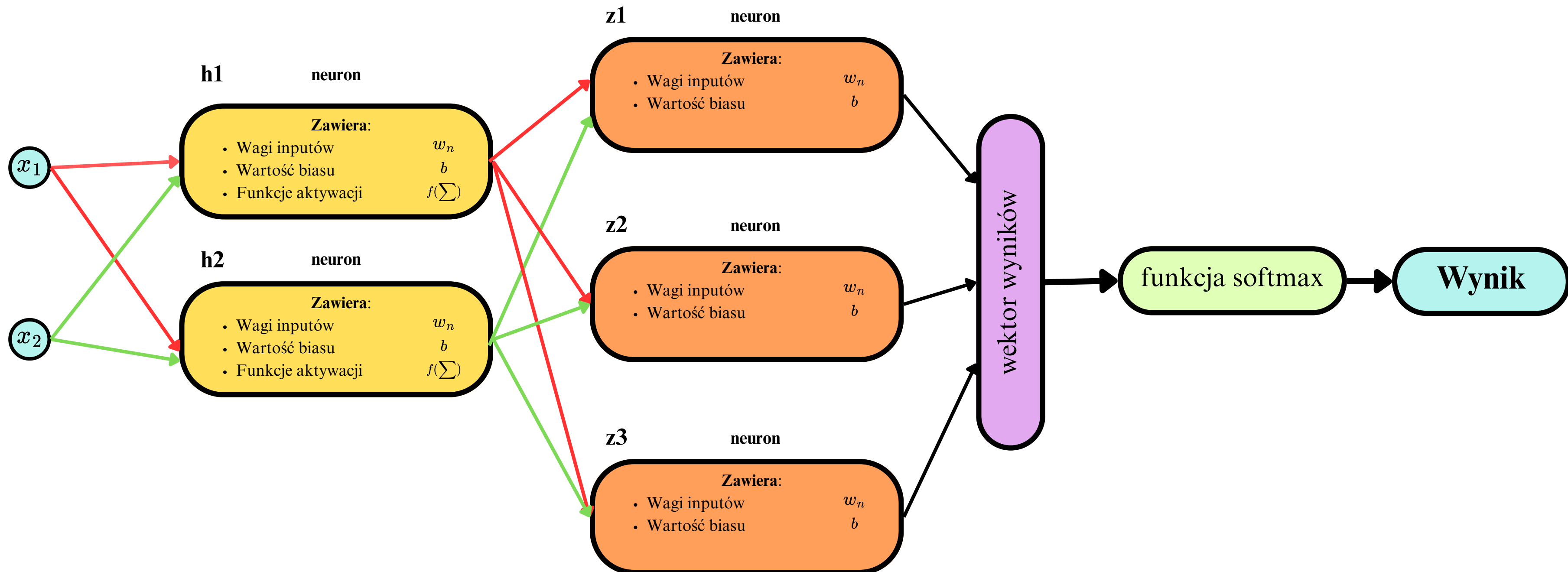
$$x_1, x_2 \rightarrow h_1, h_2 \rightarrow z_1, z_2, z_3 \rightarrow \text{softmax} \rightarrow \text{wynik}$$

input

Warstwa ukryta

Neurony warstwy wyjściowej

output



Macierze w sieciach neuronowych

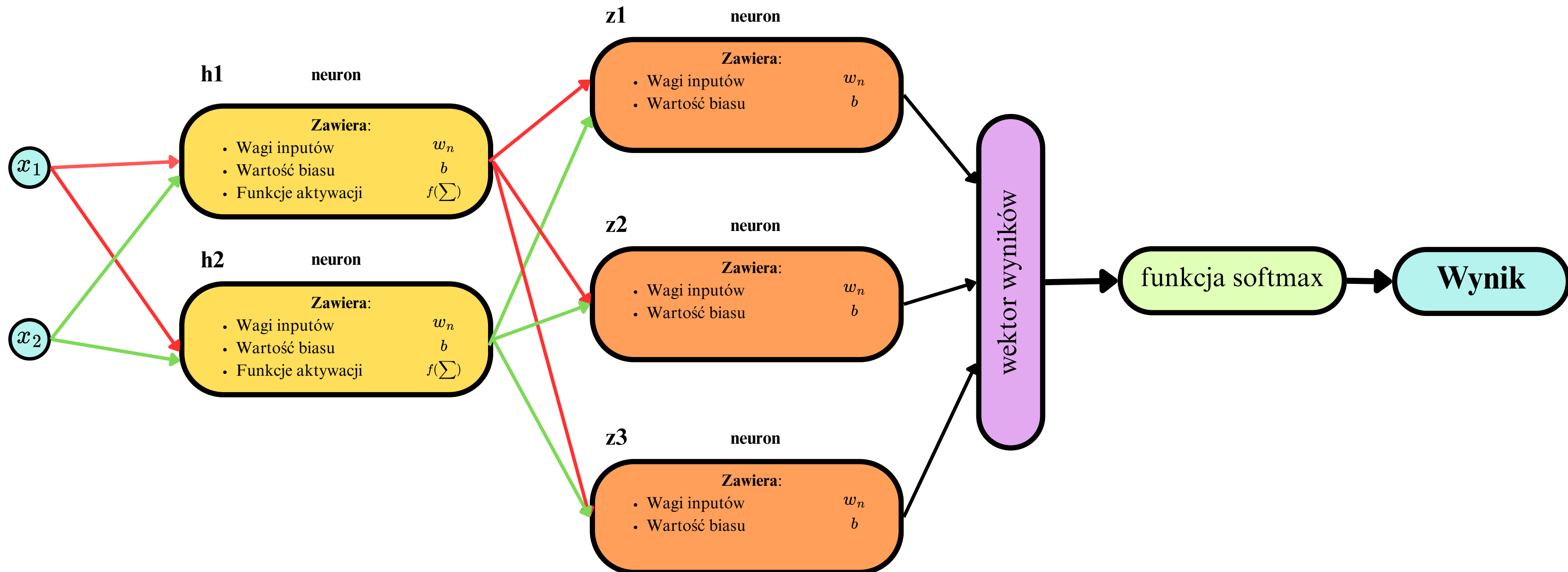
$$x_1, x_2 \rightarrow h_1, h_2 \rightarrow z_1, z_2, z_3 \rightarrow \text{softmax} \rightarrow \text{wynik}$$

input

Warstwa ukryta

Neurony warstwy wyjściowej

output

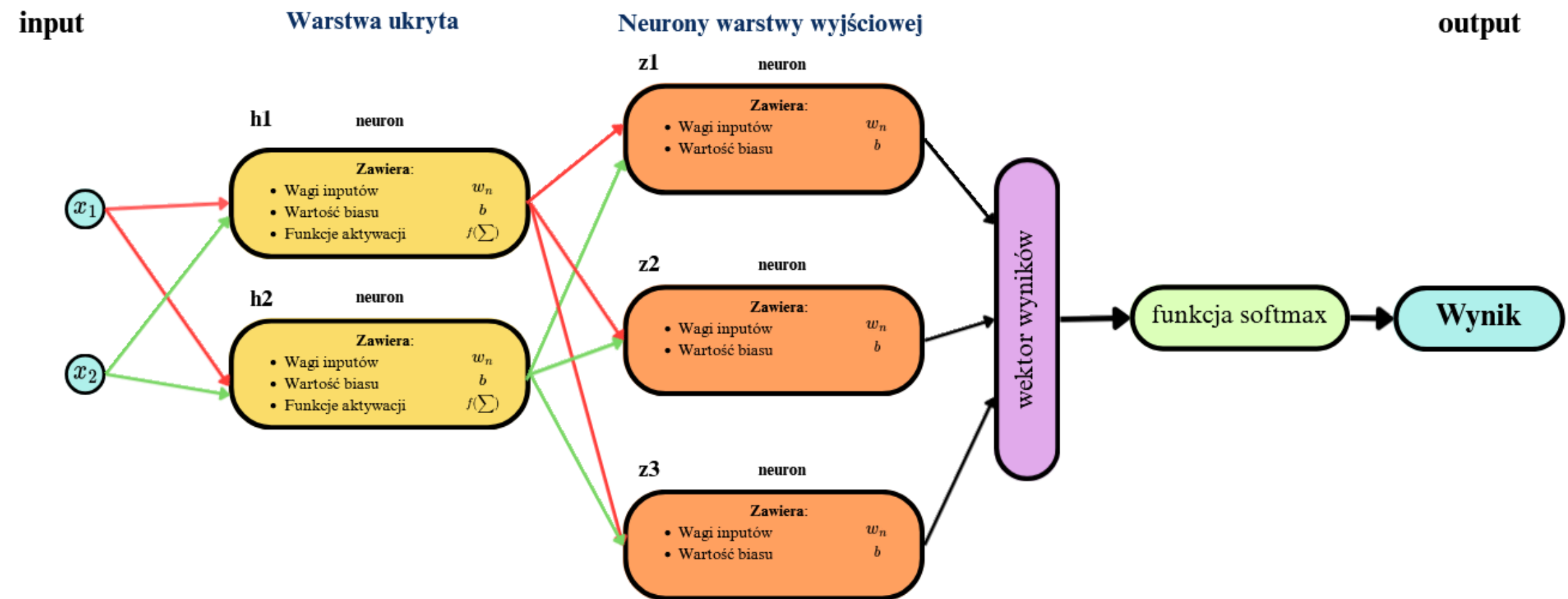


Macierze w sieciach neuronowych

$$x_1, x_2 \rightarrow h_1, h_2 \rightarrow z_1, z_2, z_3 \rightarrow softmax \rightarrow wynik$$

Wyobraźmy sobie, że chcemy napisać kod, który w oparciu o input, wagi, biasy oraz f. softmax będzie podawał nam propozycje klasy. Kod wyglądałby najpewniej w ten sposób:

- wpisz input
- policz wektor wartości zwracany przez warstwę ukrytą
- policz wektor wartości zwracany przez warstwę wyjściową
- użyj funkcji softmax na wektorze wyników
- wybierz klasę o najwyższym prawdopodobieństwie



Macierze w sieciach neuronowych



Wydział
Biologii

$$x_1, x_2 \rightarrow h_1, h_2 \rightarrow z_1, z_2, z_3 \rightarrow \text{softmax} \rightarrow \text{wynik}$$

Każdy neuron warstwy ukrytej liczy:

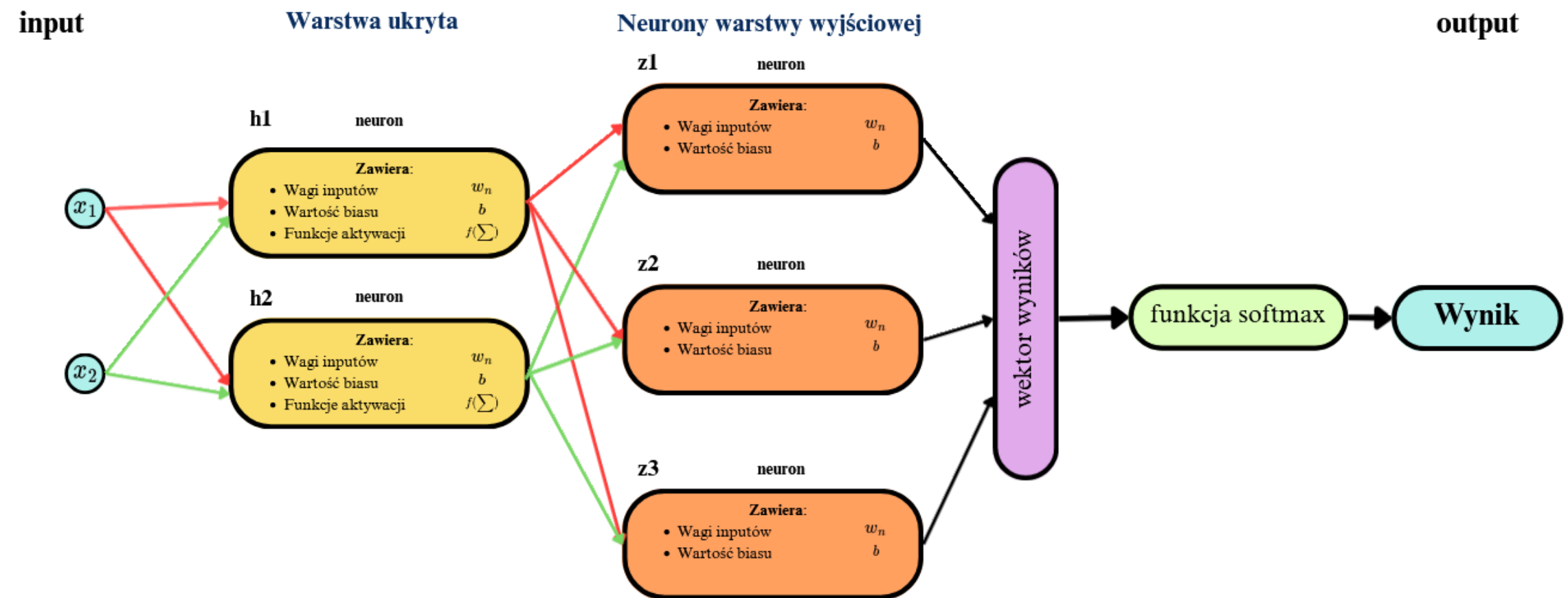
$$y = f(w_1x_1 + w_2x_2 + b)$$

Każdy neuron warstwy wyjściowej liczy:

$$z = w_1x_1 + w_2x_2 + b_1$$

Czyli cała warstwa to:

$$\begin{cases} z_1 = w_{11}x_1 + w_{12}x_2 + b_1 \\ z_2 = w_{21}x_1 + w_{22}x_2 + b_2 \\ z_3 = w_{31}x_1 + w_{32}x_2 + b_3 \end{cases}$$



Macierze w sieciach neuronowych

$$x_1, x_2 \rightarrow h_1, h_2 \rightarrow z_1, z_2, z_3 \rightarrow \textit{softmax} \rightarrow \textit{wynik}$$

Każdy neuron warstwy ukrytej liczy:

$$y = f(w_1x_1 + w_2x_2 + b)$$

Licząc to krok po kroku tworzylibyśmy bardzo długi kod o wielu pętlach.

Czy istnieje prostszy sposób?

$$z = Wx + b$$

Każdy neuron warstwy wyjściowej liczy:

$$z = w_1x_1 + w_2x_2 + b_1$$

Czyli cała warstwa to:

$$\begin{cases} z_1 = w_{11}x_1 + w_{12}x_2 + b_1 \\ z_2 = w_{21}x_1 + w_{22}x_2 + b_2 \\ z_3 = w_{31}x_1 + w_{32}x_2 + b_3 \end{cases}$$

Macierze w sieciach neuronowych

$$x_1, x_2 \rightarrow h_1, h_2 \rightarrow z_1, z_2, z_3 \rightarrow \text{softmax} \rightarrow \text{wynik}$$

Każdy neuron warstwy ukrytej liczy:

$$y = f(w_1x_1 + w_2x_2 + b)$$

Licząc to krok po kroku tworzylibyśmy bardzo długi kod o wielu pętlach.

Czy istnieje prostszy sposób?

$$z = Wx + b$$

Każdy neuron warstwy wyjściowej liczy:

$$z = w_1x_1 + w_2x_2 + b_1$$

Czyli cała warstwa to:

$$\begin{cases} z_1 = w_{11}x_1 + w_{12}x_2 + b_1 \\ z_2 = w_{21}x_1 + w_{22}x_2 + b_2 \\ z_3 = w_{31}x_1 + w_{32}x_2 + b_3 \end{cases}$$

x

$$\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

$$\textcircled{x_1}$$

$$\textcircled{x_2}$$

Macierze w sieciach neuronowych

$$x_1, x_2 \rightarrow h_1, h_2 \rightarrow z_1, z_2, z_3 \rightarrow \text{softmax} \rightarrow \text{wynik}$$

Każdy neuron warstwy ukrytej liczy:

$$y = f(w_1x_1 + w_2x_2 + b)$$

Licząc to krok po kroku tworzylibyśmy bardzo długi kod o wielu pętlach.

Czy istnieje prostszy sposób?

$$z = Wx + b$$

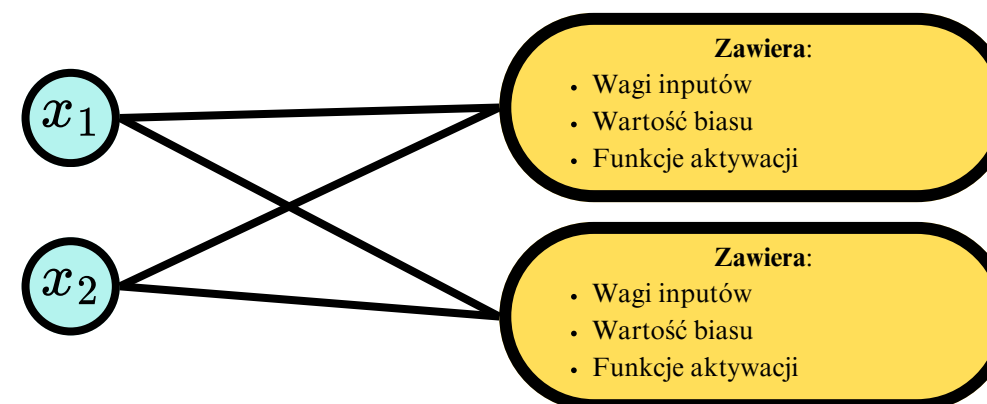
Każdy neuron warstwy wyjściowej liczy:

$$z = w_1x_1 + w_2x_2 + b_1$$

Czyli cała warstwa to:

$$\begin{cases} z_1 = w_{11}x_1 + w_{12}x_2 + b_1 \\ z_2 = w_{21}x_1 + w_{22}x_2 + b_2 \\ z_3 = w_{31}x_1 + w_{32}x_2 + b_3 \end{cases}$$

$$\begin{matrix} x & W & b & x \end{matrix}$$
$$\begin{vmatrix} x_1 \\ x_2 \end{vmatrix} \begin{vmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{vmatrix} \begin{vmatrix} b_1 \\ b_2 \end{vmatrix} \rightarrow \begin{vmatrix} x_1 \\ x_2 \end{vmatrix}$$



Macierze w sieciach neuronowych

$$x_1, x_2 \rightarrow h_1, h_2 \rightarrow z_1, z_2, z_3 \rightarrow \text{softmax} \rightarrow \text{wynik}$$

Każdy neuron warstwy ukrytej liczy:

$$y = f(w_1x_1 + w_2x_2 + b)$$

Licząc to krok po kroku tworzylibyśmy bardzo długi kod o wielu pętlach.

Czy istnieje prostszy sposób?

$$z = Wx + b$$

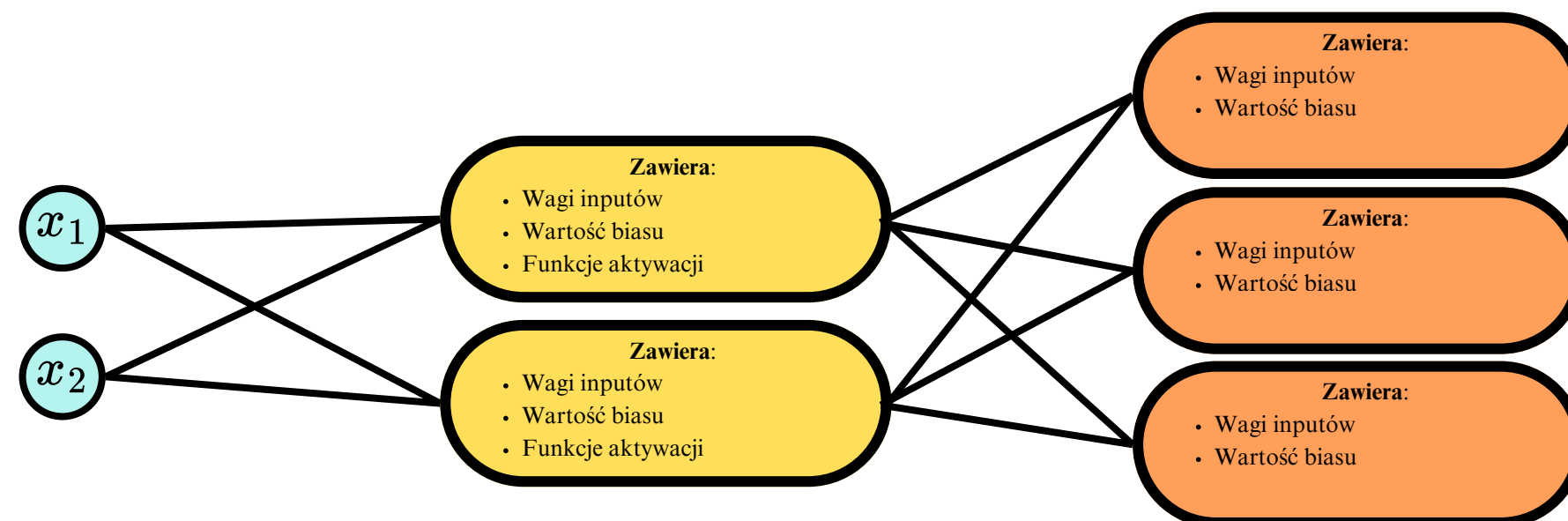
Każdy neuron warstwy wyjściowej liczy:

$$z = w_1x_1 + w_2x_2 + b_1$$

Czyli cała warstwa to:

$$\begin{cases} z_1 = w_{11}x_1 + w_{12}x_2 + b_1 \\ z_2 = w_{21}x_1 + w_{22}x_2 + b_2 \\ z_3 = w_{31}x_1 + w_{32}x_2 + b_3 \end{cases}$$

x	W	b	x	W	b	z
$\begin{vmatrix} x_1 \\ x_2 \end{vmatrix}$	$\begin{vmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{vmatrix}$	$\begin{vmatrix} b_1 \\ b_2 \end{vmatrix}$	$\begin{vmatrix} x_1 \\ x_2 \end{vmatrix}$	$\begin{vmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \\ w_{31} & w_{32} \end{vmatrix}$	$\begin{vmatrix} b_1 \\ b_2 \\ b_3 \end{vmatrix}$	$\begin{vmatrix} z_1 \\ z_2 \\ z_3 \end{vmatrix}$



Macierze w sieciach neuronowych

$$x_1, x_2 \rightarrow h_1, h_2 \rightarrow z_1, z_2, z_3 \rightarrow \text{softmax} \rightarrow \text{wynik}$$

Każdy neuron warstwy ukrytej liczy:

$$y = f(w_1x_1 + w_2x_2 + b)$$

Licząc to krok po kroku tworzylibyśmy bardzo długi kod o wielu pętlach.

Czy istnieje prostszy sposób?

$$z = Wx + b$$

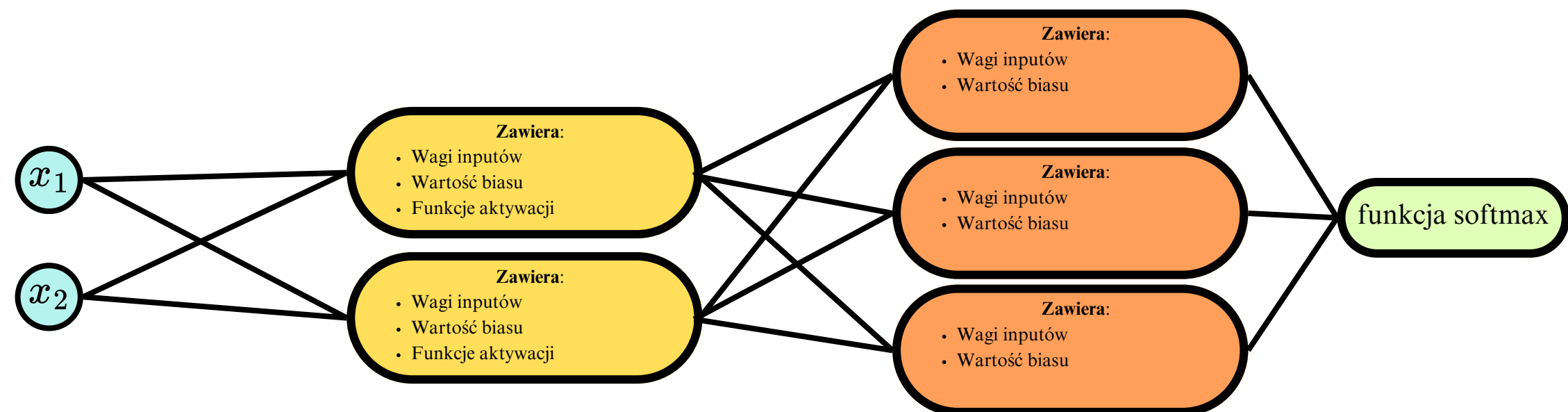
Każdy neuron warstwy wyjściowej liczy:

$$z = w_1x_1 + w_2x_2 + b_1$$

Czyli cała warstwa to:

$$\begin{cases} z_1 = w_{11}x_1 + w_{12}x_2 + b_1 \\ z_2 = w_{21}x_1 + w_{22}x_2 + b_2 \\ z_3 = w_{31}x_1 + w_{32}x_2 + b_3 \end{cases}$$

x	W	b	x	W	b	z	p
$\begin{vmatrix} x_1 \\ x_2 \end{vmatrix}$	$\begin{vmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{vmatrix}$	$\begin{vmatrix} b_1 \\ b_2 \end{vmatrix}$	$\begin{vmatrix} x_1 \\ x_2 \end{vmatrix}$	$\begin{vmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \\ w_{31} & w_{32} \end{vmatrix}$	$\begin{vmatrix} b_1 \\ b_2 \\ b_3 \end{vmatrix}$	$\begin{vmatrix} z_1 \\ z_2 \\ z_3 \end{vmatrix}$	$\begin{vmatrix} p_1 \\ p_2 \\ p_3 \end{vmatrix}$



$$\begin{vmatrix} 7 & 2 \\ 1 & 9 \end{vmatrix} \quad \begin{vmatrix} 2 & 3 \\ 2 & -2 \\ 4 & 4 \end{vmatrix} \quad \begin{vmatrix} 9 & 3 & 4 \\ 1 & 5 & 5 \end{vmatrix}$$

Macierzą nazywamy prostokątną tablicę o **m wierszach** i **n kolumnach** oraz o **elementach ciała K**. Macierze oznaczamy dużymi literami. Elementy macierzy nazywamy wyrazami.

matrixcalc.org

Macierze możemy:

- **Dodawać/Odejmować** (wykonywalne dla macierzy o tych samych wymiarach)
- **Mnożyć przez liczbę** (skalar) - każdy wyraz mnożymy przez skalar
- **Mnożyć przez inną matrycę**

$$A_{3 \times 2}(R) = \begin{vmatrix} 2 & 9 \\ 7 & -2 \\ 1 & 4 \end{vmatrix}$$

$$B_{3 \times 1}(R) = \begin{vmatrix} 3 \\ 2 \\ 3 \end{vmatrix}$$

$$\begin{vmatrix} 7 & 2 \\ 1 & 9 \end{vmatrix} \quad \begin{vmatrix} 2 & 3 \\ 2 & -2 \\ 4 & 4 \end{vmatrix} \quad \begin{vmatrix} 9 & 3 & 4 \\ 1 & 5 & 5 \end{vmatrix}$$

Macierzą nazywamy prostokątną tablicę o **m wierszach** i **n kolumnach** oraz o **elementach ciała K**. Macierze oznaczamy dużymi literami. Elementy macierzy nazywamy wyrazami.

matrixcalc.org

Macierze możemy:

- **Dodawać/Odejmować** (wykonywalne dla macierzy o tych samych wymiarach)
- **Mnożyć przez liczbę** (skalar) - każdy wyraz mnożymy przez skalar
- **Mnożyć przez inną macierz**

$$A_{3 \times 2}(R) = \begin{vmatrix} 2 & 9 \\ 7 & -2 \\ 1 & 4 \end{vmatrix} \quad B_{3 \times 1}(R) = \begin{vmatrix} 3 \\ 2 \\ 3 \end{vmatrix} \quad \vec{v} = (3, 2, 3) = \begin{pmatrix} 3 \\ 2 \\ 3 \end{pmatrix}$$

wektor

Macierze - przypomnienie

Działania na macierzach



Wydział
Biologii

$$A_{2 \times 2} = \begin{vmatrix} 2 & 3 \\ 2 & -2 \end{vmatrix} \quad B_{2 \times 2} = \begin{vmatrix} 7 & 9 \\ 2 & -2 \end{vmatrix}$$

Macierze można mnożyć gdy:

- mają równe wymiary
- liczba kolumn pierwszej i liczba wierszy drugiej się zgadzają

$$A \times B = \begin{vmatrix} \textcircled{2} & \textcircled{3} \\ 2 & -2 \end{vmatrix} \times \begin{vmatrix} \textcircled{7} & 9 \\ \textcircled{2} & -2 \end{vmatrix} = \begin{vmatrix} \textcircled{-} & \textcircled{-} \\ - & - \end{vmatrix}$$

$$\begin{vmatrix} \textcircled{2} & \textcircled{3} \\ 2 & -2 \end{vmatrix} \times \begin{vmatrix} 7 & \textcircled{9} \\ 2 & \textcircled{-2} \end{vmatrix} = \begin{vmatrix} 20 & \textcircled{-} \\ - & - \end{vmatrix}$$

$$\begin{vmatrix} 2 & 3 \\ \textcircled{2} & \textcircled{-2} \end{vmatrix} \times \begin{vmatrix} \textcircled{7} & 9 \\ \textcircled{2} & -2 \end{vmatrix} = \begin{vmatrix} 20 & 12 \\ \textcircled{-} & - \end{vmatrix}$$

$$\begin{vmatrix} 2 & 3 \\ \textcircled{2} & \textcircled{-2} \end{vmatrix} \times \begin{vmatrix} 7 & \textcircled{9} \\ 2 & \textcircled{-2} \end{vmatrix} = \begin{vmatrix} 20 & 12 \\ 10 & \textcircled{-} \end{vmatrix}$$

$$\begin{pmatrix} 2 & 3 \\ 2 & -2 \end{pmatrix} \cdot \begin{pmatrix} 7 & 9 \\ 2 & -2 \end{pmatrix} = \begin{pmatrix} 2 \cdot 7 + 3 \cdot 2 & 2 \cdot 9 + 3 \cdot (-2) \\ 2 \cdot 7 + (-2) \cdot 2 & 2 \cdot 9 + (-2) \cdot (-2) \end{pmatrix} = \begin{pmatrix} 20 & 12 \\ 10 & 22 \end{pmatrix}$$

$$\begin{vmatrix} \textcircled{2} & \textcircled{3} \\ 2 & -2 \end{vmatrix} \times \begin{vmatrix} \textcircled{7} & 9 \\ \textcircled{2} & -2 \end{vmatrix} = \begin{vmatrix} \textcircled{-} & \textcircled{-} \\ - & - \end{vmatrix}$$



$$A_{3 \times 2} = \begin{vmatrix} 2 & 3 \\ 2 & -2 \\ 4 & 4 \end{vmatrix} \quad B_{2 \times 4} = \begin{vmatrix} 1 & 3 & 2 & 4 \\ 2 & 1 & -2 & 3 \end{vmatrix}$$

Transponowanie macierzy
(Zamiana kolumn z wierszami)

$$A_{3 \times 2} = \begin{vmatrix} 2 & 3 \\ 2 & -2 \\ 4 & 4 \end{vmatrix} \quad A^T = \begin{vmatrix} 2 & 2 & 4 \\ 3 & -2 & 4 \end{vmatrix}$$

$$(A^T)^T = A$$

$$A = \begin{bmatrix} a & b & c \\ d & e & f \end{bmatrix}_{2 \times 3}$$

$$A^T = \begin{bmatrix} a & d \\ b & e \\ c & f \end{bmatrix}_{3 \times 2}$$

Do zapamiętania

Mnożenie wektora przez macierz to nie tylko operacja matematyczna, lecz przekształcenie przestrzeni. Gdy mnożymy wektor wejściowy X przez macierz W , przenosimy punkt z jednej przestrzeni do innej.

- **Kolejność mnożenia macierzy ma znaczenie!**
- **By macierze można było przez siebie mnożyć, muszą mieć one odpowiednie wymiary!**

W zależności od wartości w macierzy, to przekształcenie może rozciągać, obracać, odbijać lub ściskać przestrzeń. W sieci neuronowej każda warstwa wykonuje dokładnie taką operację. Macierz W_1 bierze punkt (x_1, x_2) i przenosi go do nowej przestrzeni (h_1, h_2) , w której łatwiej dostrzec różnice między klasami. Następnie funkcja aktywacji "zagina" tę przestrzeń ucinając ujemne wartości, a kolejna macierz W_2 przekształca ją ponownie itd. Finalna macierz W_x szuka granic między klasami. Cały trening sieci neuronowej to nic innego jak uczenie się takich macierzy, które przekształcają przestrzeń w sposób umożliwiający rozdzielenie danych różnych klas, nawet jeśli w oryginalnej przestrzeni było to niemożliwe

Mnożenie macierzy w Python



Wydział
Biologii

```
1 # Obliczanie z wykorzystaniem macierzy
2
3 import numpy as np
4
5 W1 = np.array([
6     [1.5028, -0.7856],
7     [-0.0159, -0.0482]
8 ])
9
10 X = np.array([[8], [9]])
11
12 wynik = W1 @ X # operator @ to mnożenie macierzy
13 print(wynik)
14
15
16 # Obliczanie "na piechotę"
17 def matmul(W, X):
18     return [sum(W[i][j]*X[j] for j in range(len(X))) for i in range(len(W))]
19
20 # NUMPY:
21 W @ X
```

Macierze w sieciach neuronowych



Wydział
Biologii

$x_1, x_2 \rightarrow h_1, h_2 \rightarrow z_1, z_2, z_3 \rightarrow softmax \rightarrow wynik$

$$\mathbf{x} = \begin{bmatrix} 2 \\ 5 \end{bmatrix}$$

$$W^{(1)} = \begin{bmatrix} 0.5 & -0.3 \\ 0.8 & 0.2 \end{bmatrix}$$

$$\mathbf{h} = W^{(1)}\mathbf{x} + b^{(1)}$$

$$b^{(1)} = \begin{bmatrix} 0.1 \\ -0.2 \end{bmatrix}$$

Macierze w sieciach neuronowych



Wydział
Biologii

$x_1, x_2 \rightarrow h_1, h_2 \rightarrow z_1, z_2, z_3 \rightarrow softmax \rightarrow wynik$

$$\mathbf{x} = \begin{bmatrix} 2 \\ 5 \end{bmatrix}$$

$$W^{(1)} = \begin{bmatrix} 0.5 & -0.3 \\ 0.8 & 0.2 \end{bmatrix}$$

$$\mathbf{h} = W^{(1)}\mathbf{x} + b^{(1)}$$



$$b^{(1)} = \begin{bmatrix} 0.1 \\ -0.2 \end{bmatrix}$$

$$W^{(1)}\mathbf{x} = \begin{bmatrix} 0.5 & -0.3 \\ 0.8 & 0.2 \end{bmatrix} \begin{bmatrix} 2 \\ 5 \end{bmatrix} = \begin{bmatrix} 0.5 \cdot 2 + (-0.3) \cdot 5 \\ 0.8 \cdot 2 + 0.2 \cdot 5 \end{bmatrix} = \begin{bmatrix} 1 - 1.5 \\ 1.6 + 1 \end{bmatrix} = \begin{bmatrix} -0.5 \\ 2.6 \end{bmatrix}$$

Dodajemy bias:

$$\mathbf{h} = \begin{bmatrix} -0.5 \\ 2.6 \end{bmatrix} + \begin{bmatrix} 0.1 \\ -0.2 \end{bmatrix} = \begin{bmatrix} -0.4 \\ 2.4 \end{bmatrix}$$

Macierze w sieciach neuronowych



Wydział
Biologii

$x_1, x_2 \rightarrow h_1, h_2 \rightarrow z_1, z_2, z_3 \rightarrow softmax \rightarrow wynik$

$$\mathbf{x} = \begin{bmatrix} 2 \\ 5 \end{bmatrix}$$

$$W^{(1)} = \begin{bmatrix} 0.5 & -0.3 \\ 0.8 & 0.2 \end{bmatrix}$$

$$b^{(1)} = \begin{bmatrix} 0.1 \\ -0.2 \end{bmatrix}$$

$$\mathbf{h} = W^{(1)}\mathbf{x} + b^{(1)}$$

$$\mathbf{h} = \begin{bmatrix} -0.4 \\ 2.4 \end{bmatrix}$$

$$W^{(2)} = \begin{bmatrix} 0.7 & -0.5 \\ 0.2 & 0.9 \\ -0.3 & 0.4 \end{bmatrix}$$

$$b^{(2)} = \begin{bmatrix} 0.05 \\ -0.1 \\ 0.2 \end{bmatrix}$$

Macierze w sieciach neuronowych



Wydział
Biologii

$x_1, x_2 \rightarrow h_1, h_2 \rightarrow z_1, z_2, z_3 \rightarrow softmax \rightarrow wynik$

$$\mathbf{x} = \begin{bmatrix} 2 \\ 5 \end{bmatrix}$$

$$W^{(1)} = \begin{bmatrix} 0.5 & -0.3 \\ 0.8 & 0.2 \end{bmatrix}$$

$$b^{(1)} = \begin{bmatrix} 0.1 \\ -0.2 \end{bmatrix}$$

$$\mathbf{h} = W^{(1)}\mathbf{x} + b^{(1)}$$

$$\mathbf{h} = \begin{bmatrix} -0.4 \\ 2.4 \end{bmatrix}$$

$$W^{(2)} = \begin{bmatrix} 0.7 & -0.5 \\ 0.2 & 0.9 \\ -0.3 & 0.4 \end{bmatrix}$$

$$b^{(2)} = \begin{bmatrix} 0.05 \\ -0.1 \\ 0.2 \end{bmatrix}$$

$$\mathbf{z} = W^{(2)}\mathbf{h} + b^{(2)}$$

Macierze w sieciach neuronowych



Wydział
Biologii

$$x_1, x_2 \rightarrow h_1, h_2 \rightarrow z_1, z_2, z_3 \rightarrow \text{softmax} \rightarrow \text{wynik}$$

Mnożenie:

$$W^{(2)}\mathbf{h} = \begin{bmatrix} 0.7 & -0.5 \\ 0.2 & 0.9 \\ -0.3 & 0.4 \end{bmatrix} \begin{bmatrix} -0.4 \\ 2.4 \end{bmatrix}$$

Liczymy:

- $z_1 = 0.7(-0.4) + (-0.5)(2.4) = -0.28 - 1.2 = -1.48$
- $z_2 = 0.2(-0.4) + 0.9(2.4) = -0.08 + 2.16 = 2.08$
- $z_3 = -0.3(-0.4) + 0.4(2.4) = 0.12 + 0.96 = 1.08$

Dodajemy bias:

$$\mathbf{z} = \begin{bmatrix} -1.48 \\ 2.08 \\ 1.08 \end{bmatrix} + \begin{bmatrix} 0.05 \\ -0.1 \\ 0.2 \end{bmatrix} = \begin{bmatrix} -1.43 \\ 1.98 \\ 1.28 \end{bmatrix}$$

$$\mathbf{z} = W^{(2)}\mathbf{h} + b^{(2)}$$

Macierze w sieciach neuronowych



Wydział
Biologii

$x_1, x_2 \rightarrow h_1, h_2 \rightarrow z_1, z_2, z_3 \rightarrow softmax \rightarrow wynik$

$$\mathbf{x} = \begin{bmatrix} 2 \\ 5 \end{bmatrix}$$

$$W^{(1)} = \begin{bmatrix} 0.5 & -0.3 \\ 0.8 & 0.2 \end{bmatrix}$$

$$b^{(1)} = \begin{bmatrix} 0.1 \\ -0.2 \end{bmatrix}$$

$$\mathbf{h} = W^{(1)}\mathbf{x} + b^{(1)}$$

$$\mathbf{h} = \begin{bmatrix} -0.4 \\ 2.4 \end{bmatrix}$$

$$W^{(2)} = \begin{bmatrix} 0.7 & -0.5 \\ 0.2 & 0.9 \\ -0.3 & 0.4 \end{bmatrix}$$

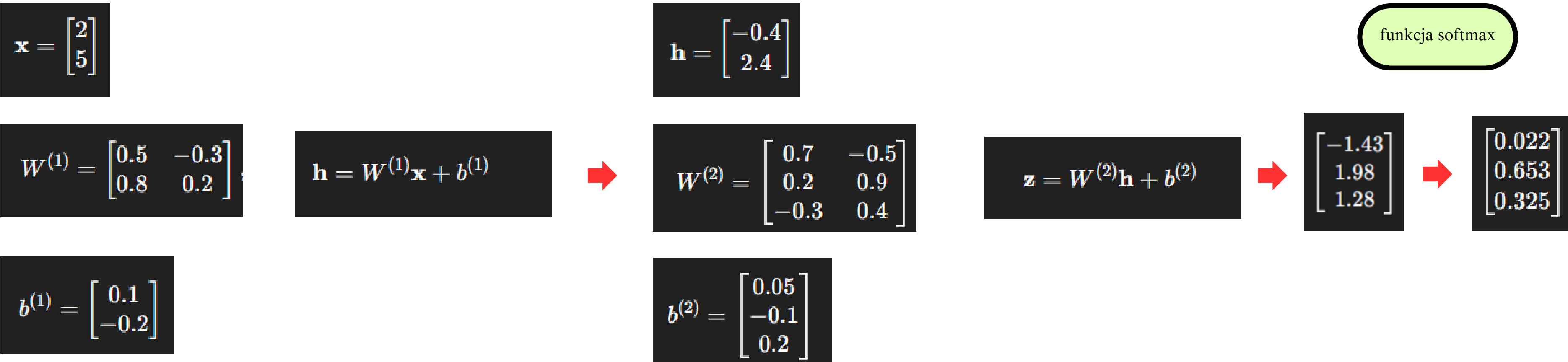
$$b^{(2)} = \begin{bmatrix} 0.05 \\ -0.1 \\ 0.2 \end{bmatrix}$$

$$\mathbf{z} = W^{(2)}\mathbf{h} + b^{(2)}$$

$$\mathbf{z} = \begin{bmatrix} -1.43 \\ 1.98 \\ 1.28 \end{bmatrix}$$

Macierze w sieciach neuronowych

$x_1, x_2 \rightarrow h_1, h_2 \rightarrow z_1, z_2, z_3 \rightarrow softmax \rightarrow wynik$



Za tydzień:



Wydział
Biologii

Macierze możemy:

- **Problemy regresji**
- **Funkcje aktywacji**
- **Forward propagation**
- **Backpropagation**
- **Uczenie sie sieci (typy danych, batche itd.)**